Bypassing ARM MTE with Speculative Execution

Jinbum Park
Security & Privacy Team
Samsung Research

Introduction

- A short story about how we started this work

- Background
  - A comparative analysis of PAC/MTE, and why MTE is more challenging to bypass.
  - Speculative probing

- Vulnerability
  - Two side-channel vulnerabilities allowing attackers the ability to leak an MTE tag

- Exploitation (in the Linux kernel)

A short story..

- Pixel8 is the first mobile device equipped with MTE? → NO! Galaxy S22 was the first one! (implemented but disabled by configuration)

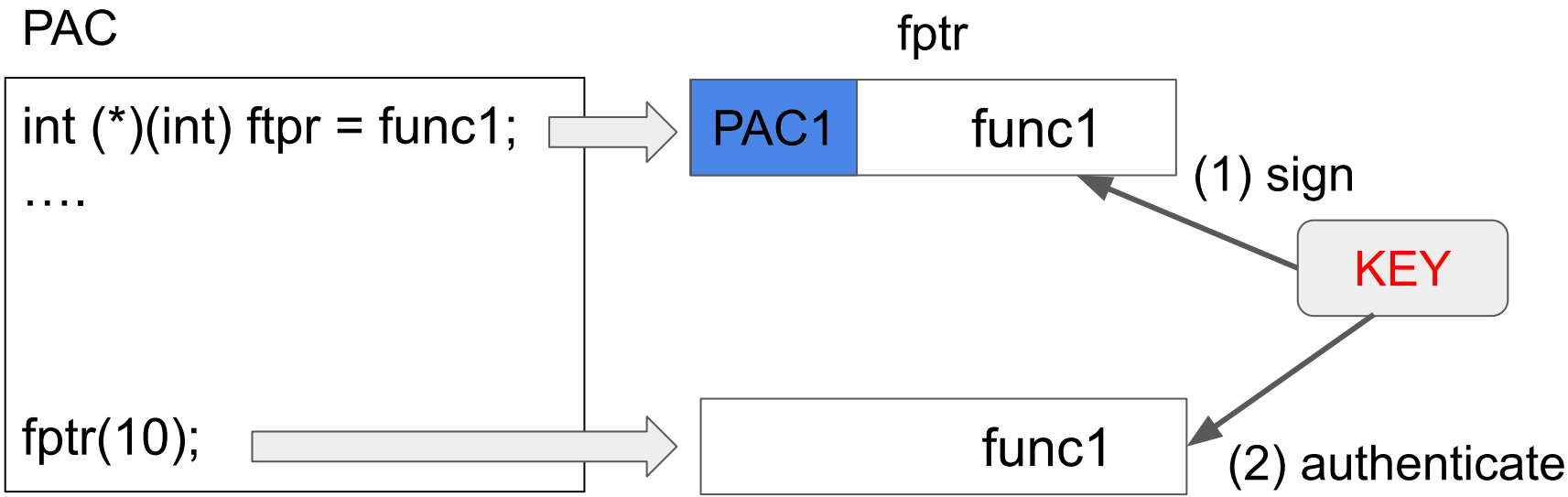
- SNU (Seoul National University) x SR (Samsung Research) collaborated for a kernel data attack mitigation that uses PAC/MTE, on S22. (before Pixel8 rolled out)

- PACMAN (ISCA 22) led us to a question: “side-channel attacks on ARM MTE are possible?”

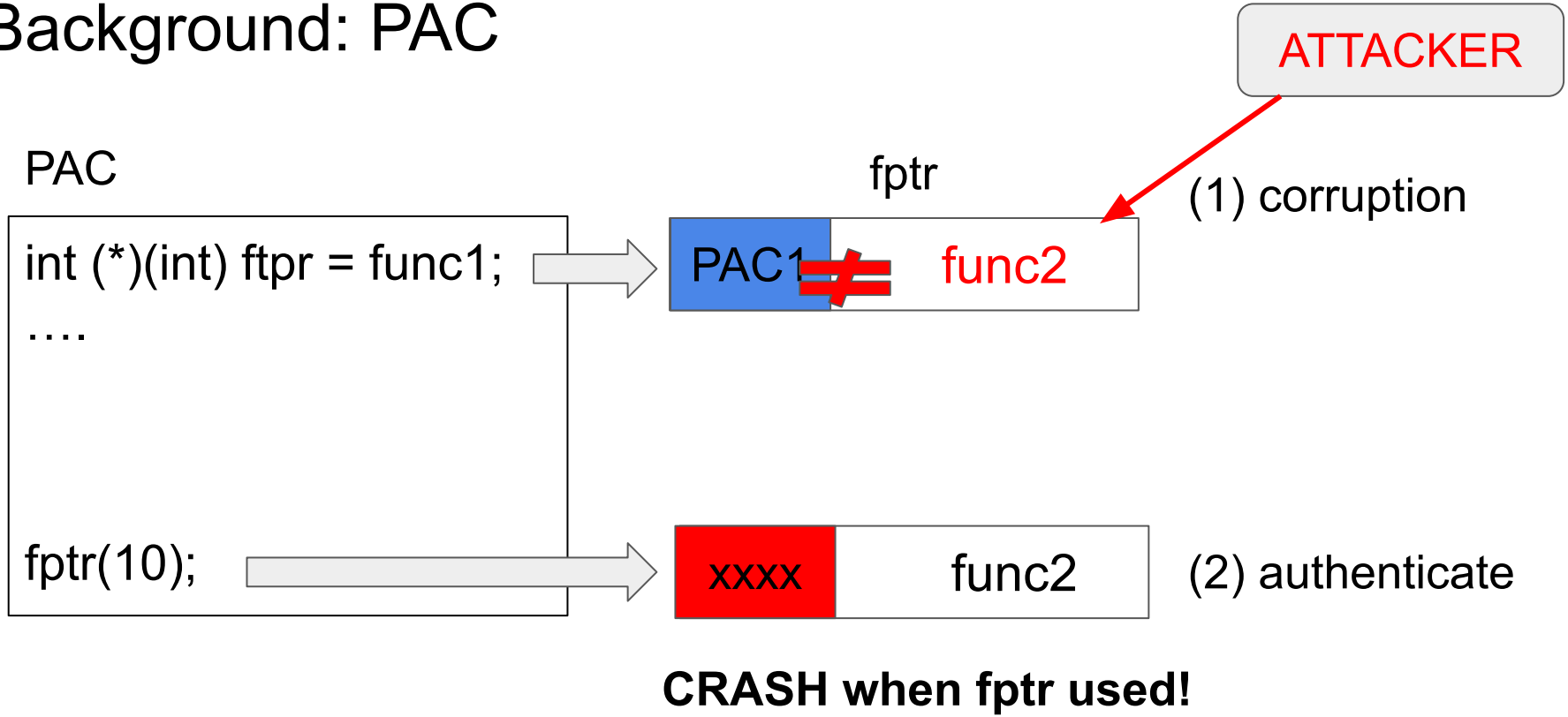
- SNU led this research in collaboration with Samsung Research (a huge credit goes to Juhee Kim et al (@SNU) for discovering these vulnerabilities)
    - Two side-channel vulnerabilities confirmed on S22 → Working on exploitations → Pixel8 rolled out → WE STARTED TO BE IN HURRY → Confirmed on Pixel8 → Reported to ARM → Working on exploitations → Paper submission (2024.02, USENIX Security, under review) → Reported to Google (confirmed as a hardware flaw of Pixel 8)

Background
(why MTE is more challenging to bypass than PAC)

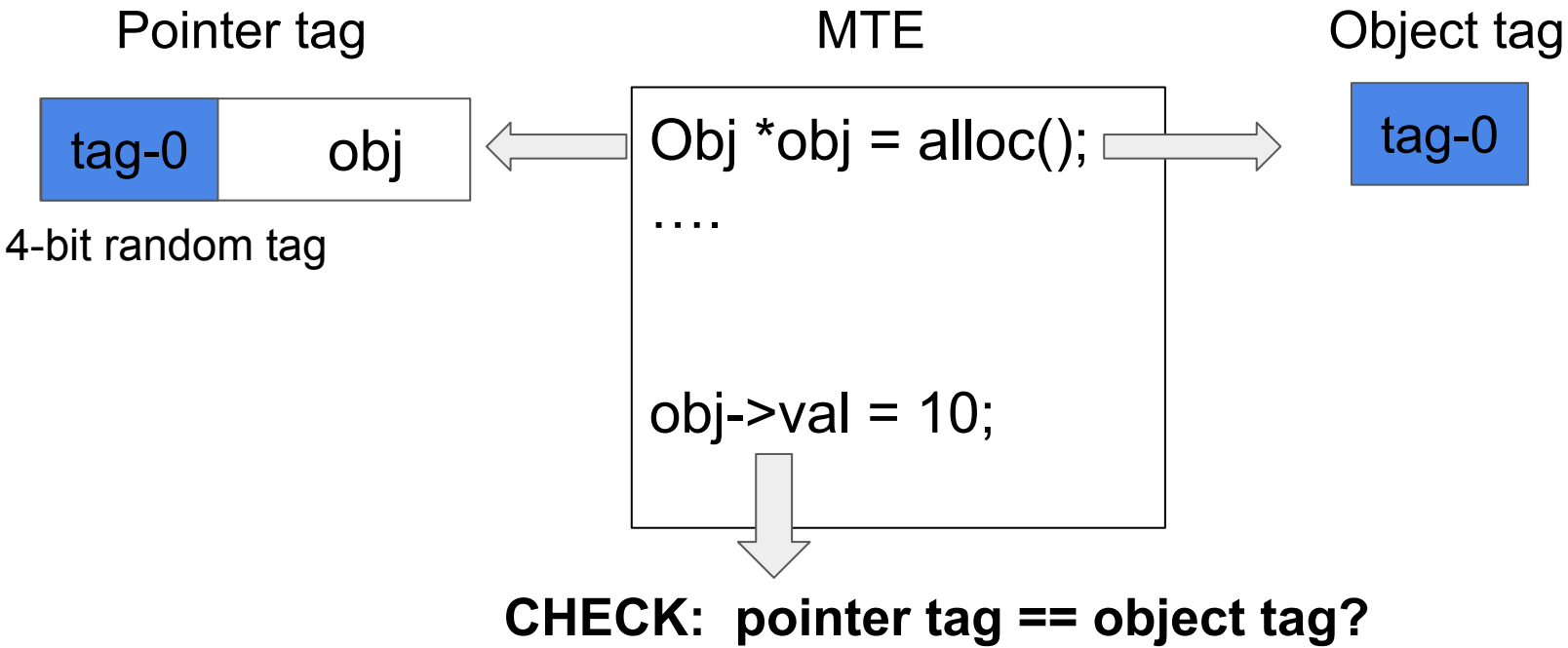
Background: PAC



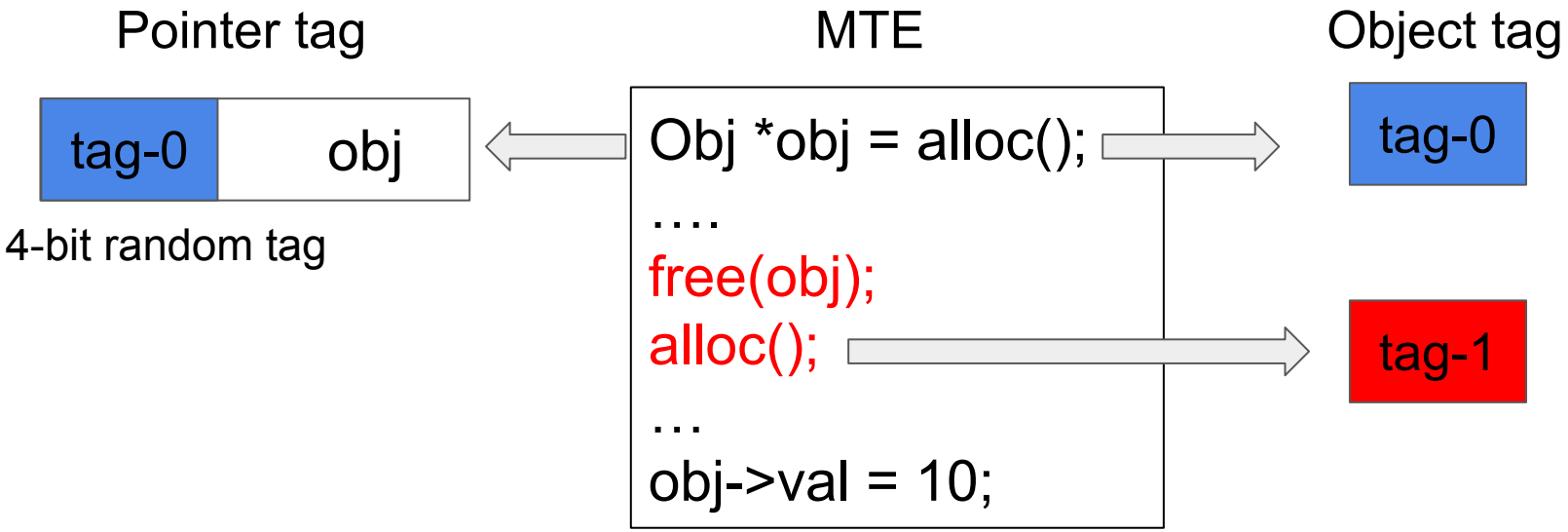
Background: PAC



Background: MTE

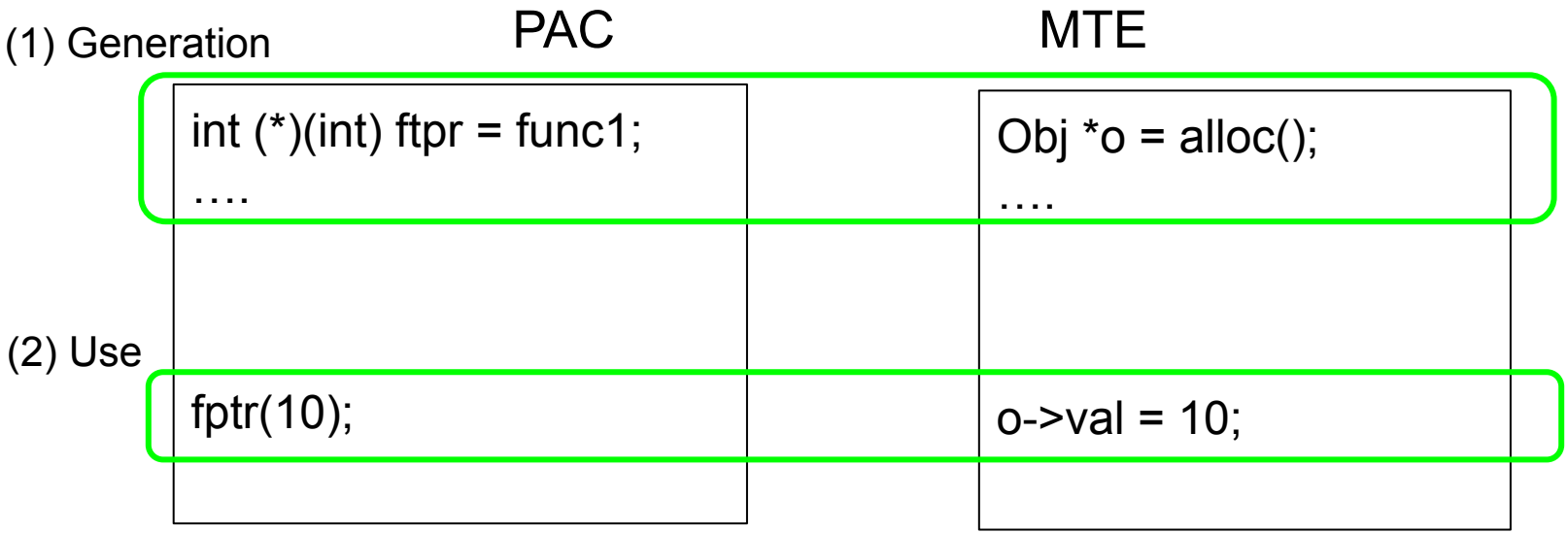


Background: MTE



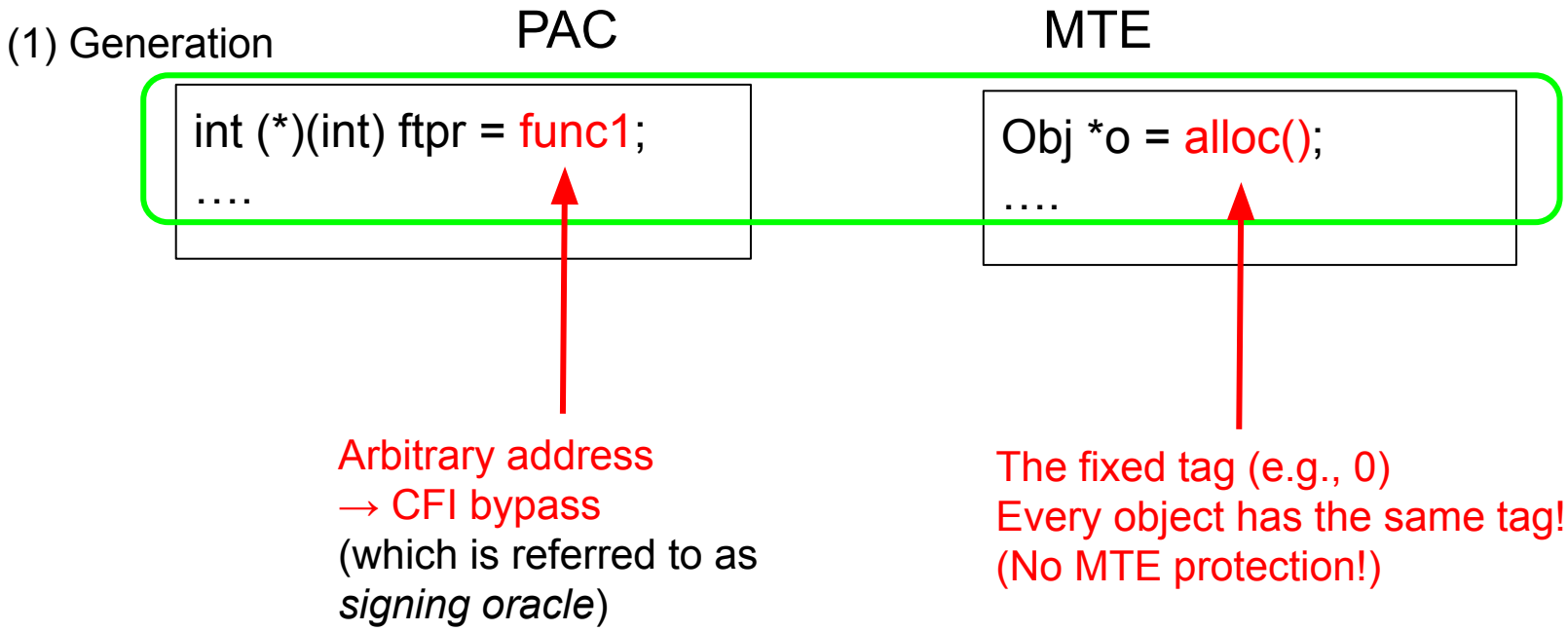
CHECK: pointer tag != object tag → CRASH!

Two phases of PAC/MTE

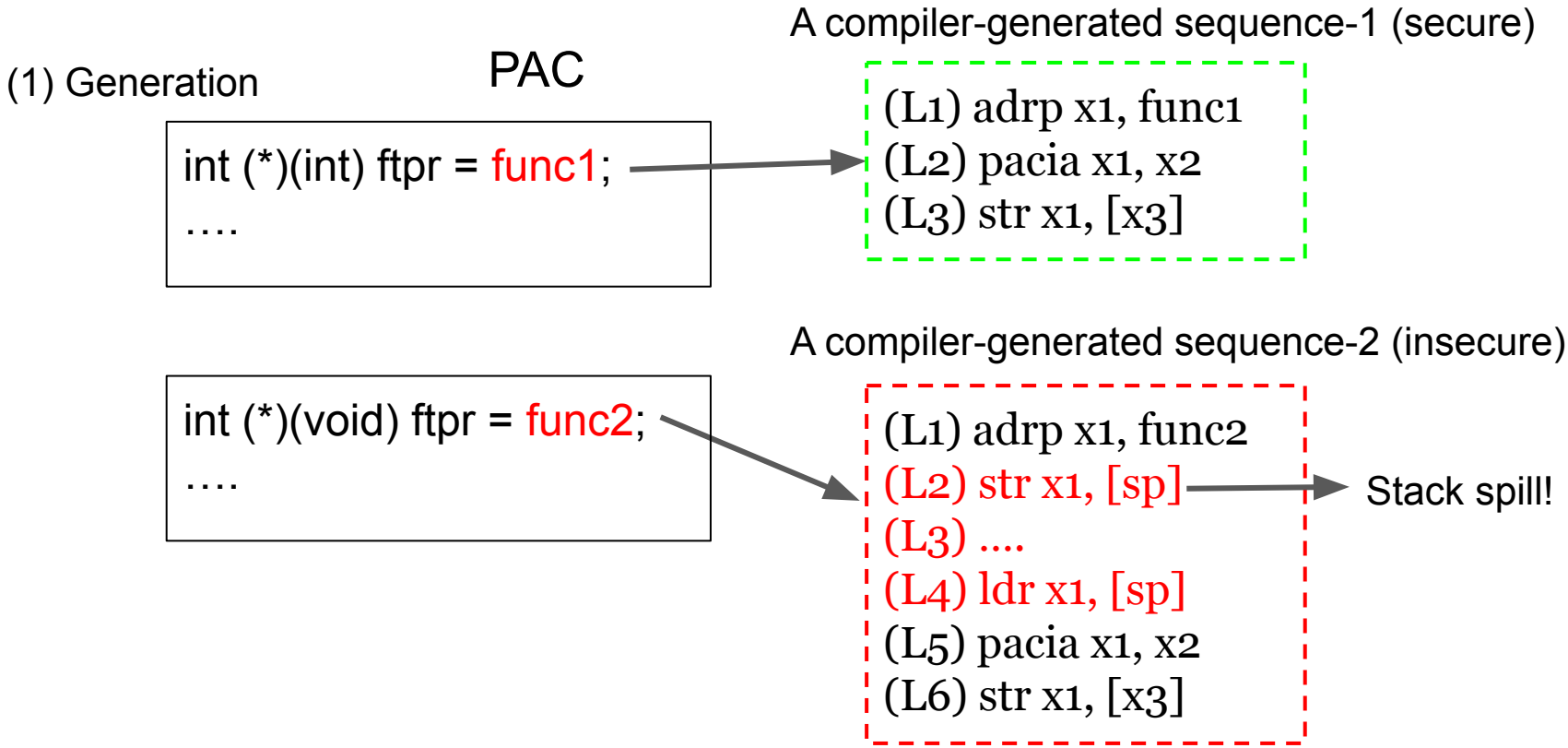


Break PAC/MTE into two common phases to make it easier to compare

Attacking the generation phase in common



Attacking the generation phase of PAC



Attacking the generation phase of PAC

A compiler-generated sequence-1 (secure)

```
(L1) adrp x1, func2
(L2) pacia x1, x2
(L3) str x1, [x3]
```

→ Most sequences are secure..

A compiler-generated sequence-2 (insecure)

```
(L1) adrp x1, func2
(L2) str x1, [sp]
(L3) ....
(L4) ldr x1, [sp]
(L5) pacia x1, x2
(L6) str x1, [x3]
```

→ But, given a large software such as linux kernel,
It's most likely you can locate insecure sequences-!

PAL (Security 22) proposed a binary analyzer that detects insecure sequences in the final kernel binary.
(Several potential signing oracle gadgets found in iOS kernel and PARTS-enabled linux)

Attacking the generation phase of MTE

(1) Generation

MTE

```
Obj *o = alloc();
….
```

A secure sequence

(1) Allocate an object
(2) irg x0 (generate a random tag)
(3) stg x0, ptr (attach the tag to the object)

An insecure sequence

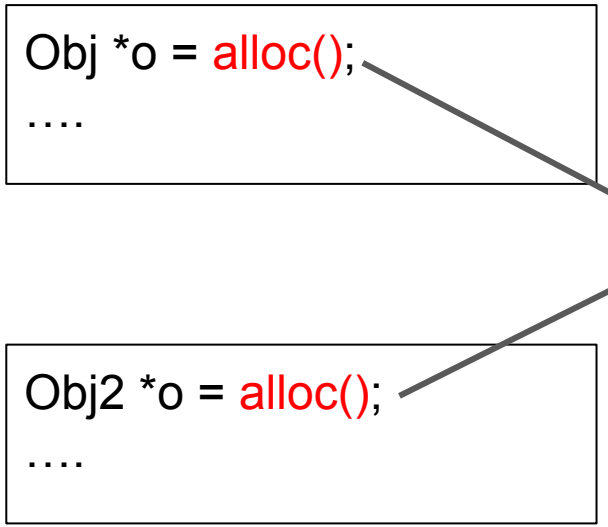
(1) Allocate an object
(2) irg x0 (generate a random tag)
(3) str x0, [sp] // stack spill
…
(4) stg x0, ptr (attach the tag to the object)

Imagine the same attack!
Is it possible??

Attacking the generation phase of MTE

(1) Generation

MTE

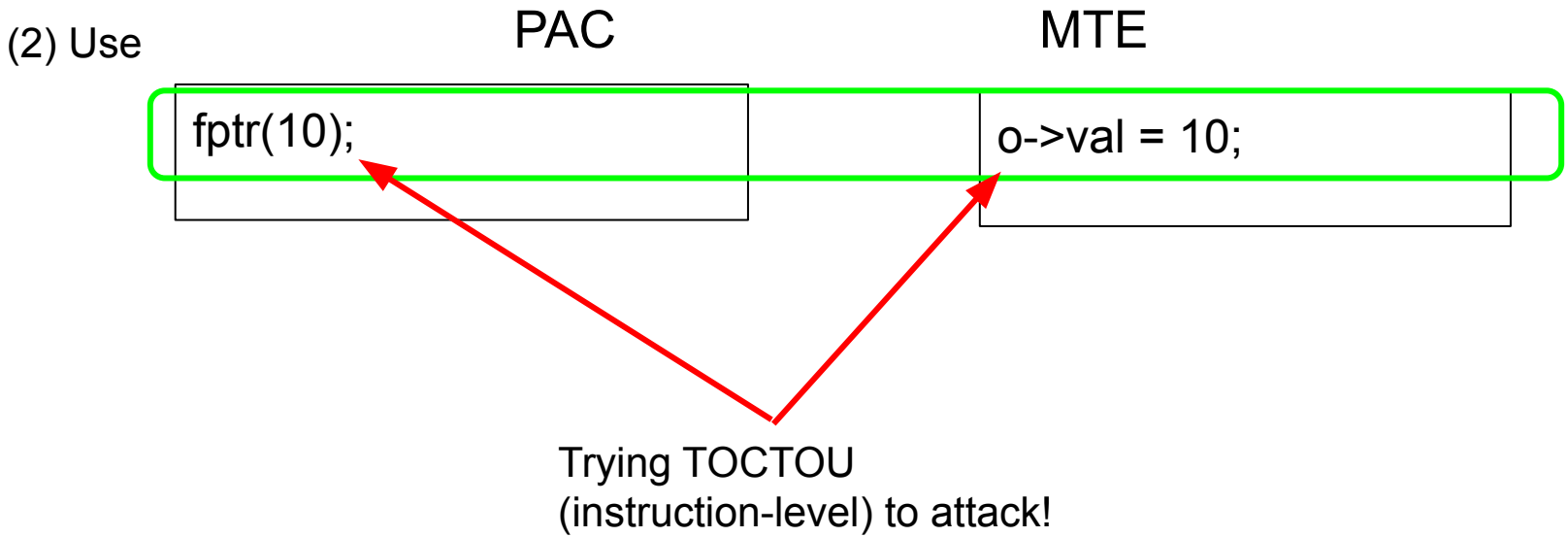


A **single** secure sequence developed and reviewed by **human experts**

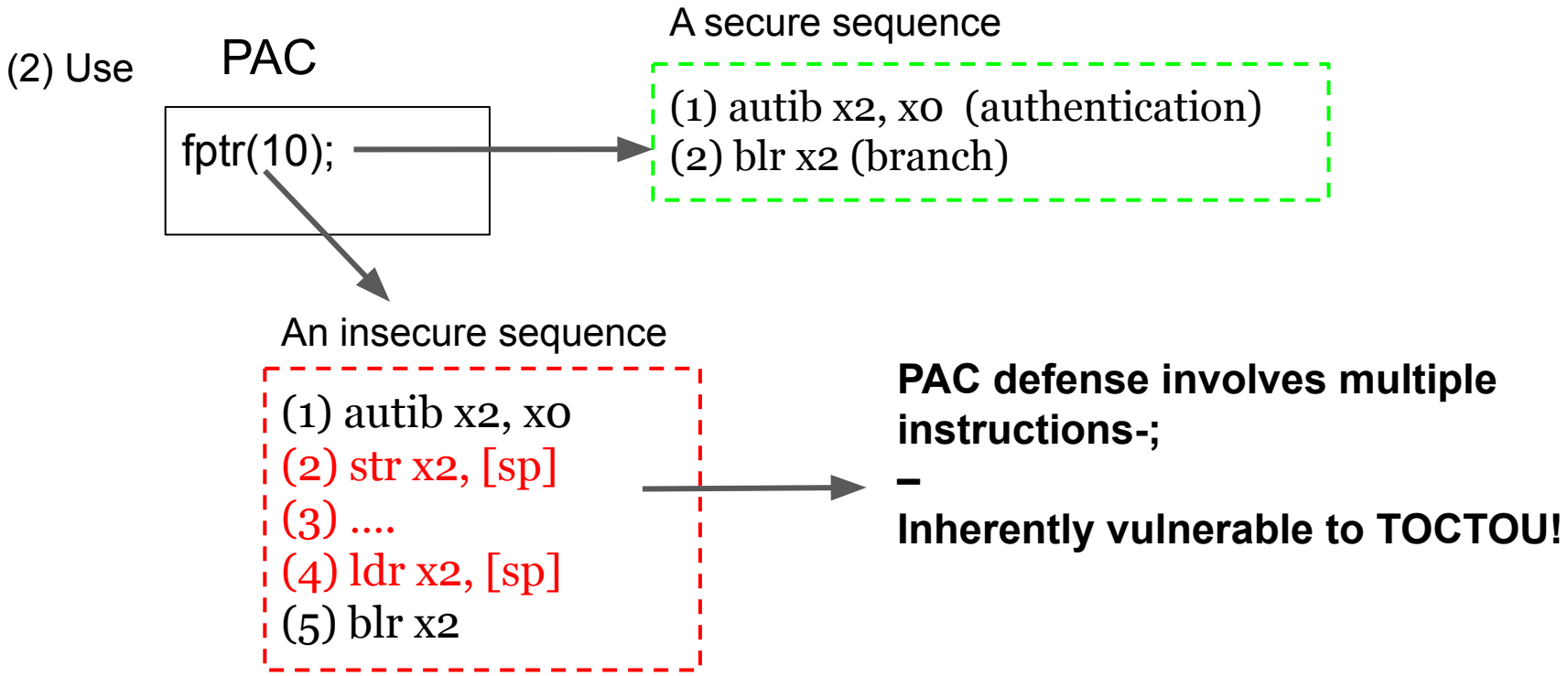
(1) Allocate an object
(2) irg x0 (generate a random tag)
(3) stg x0, ptr (attach the tag to the object)

NO possibility here….

Attacking the use phase in common

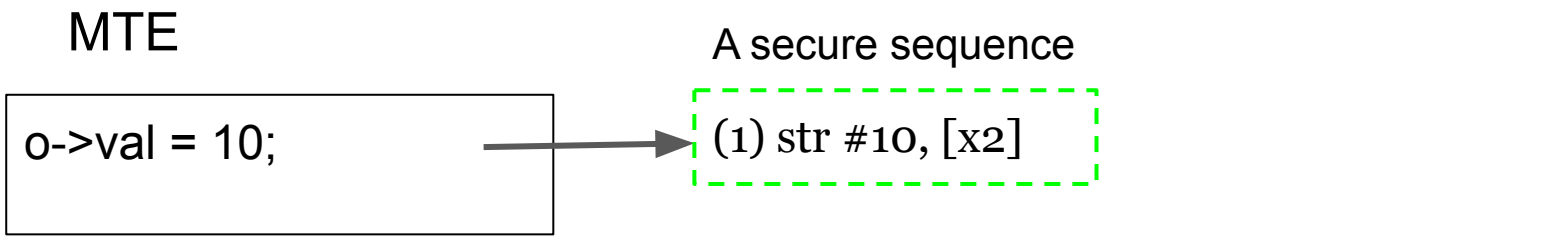


Attacking the use phase in PAC



Attacking the use phase in MTE

(2) Use

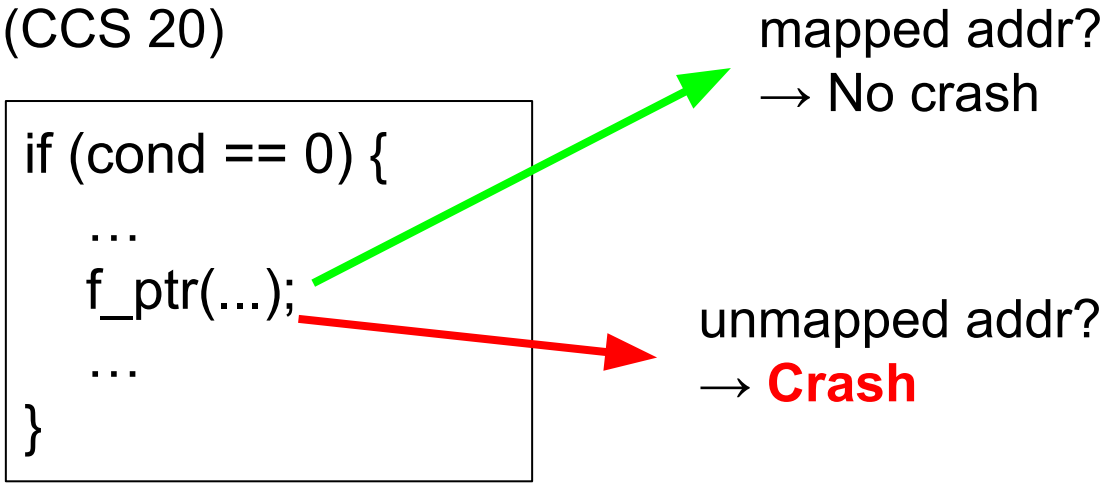


Both memory access and tag check are performed at once within one instruction!

Guaranteed by hardware!
No TOCTOU possibility here!

Side-channel attacks

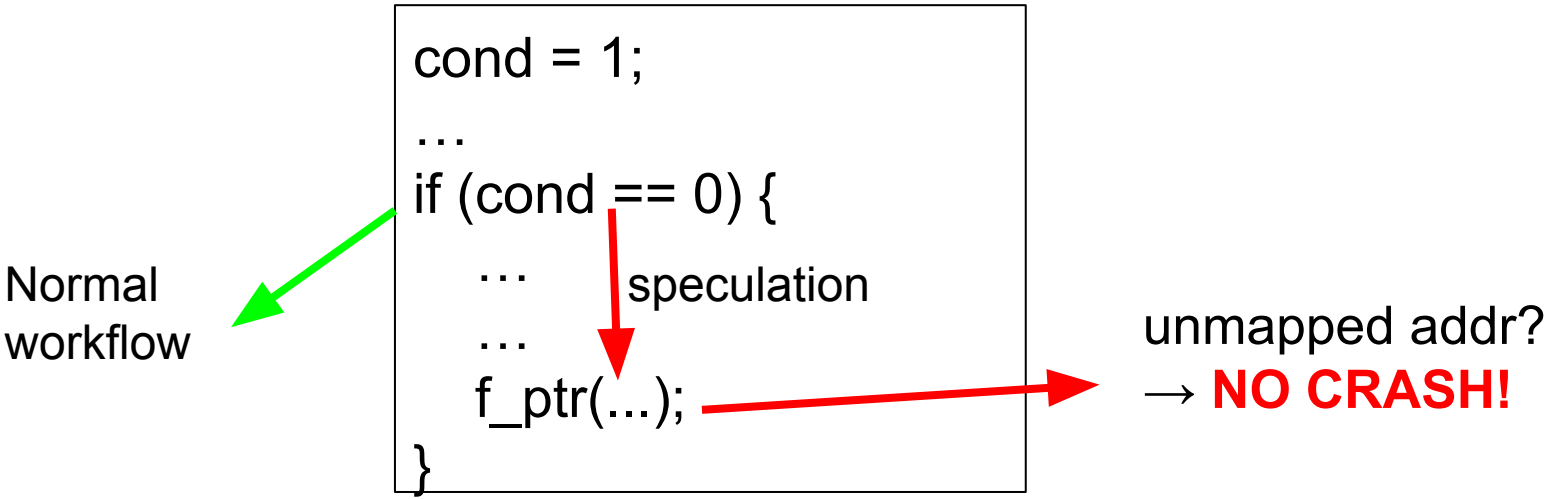
Speculative Probing (CCS 20)



- The ability we have: modifying f_ptr
- The objective: bypassing ASLR by brute-force
- The problem: putting in a wrong pointer resulting in a crash

Side-channel attacks

Speculative Probing (CCS 20)



The solution: speculatively executing f_ptr! and checking cache!

Side-channel attacks

Speculative Probing in general

PAC

```
if (expression) {
    …
    f_ptr(...);
    …
}
```

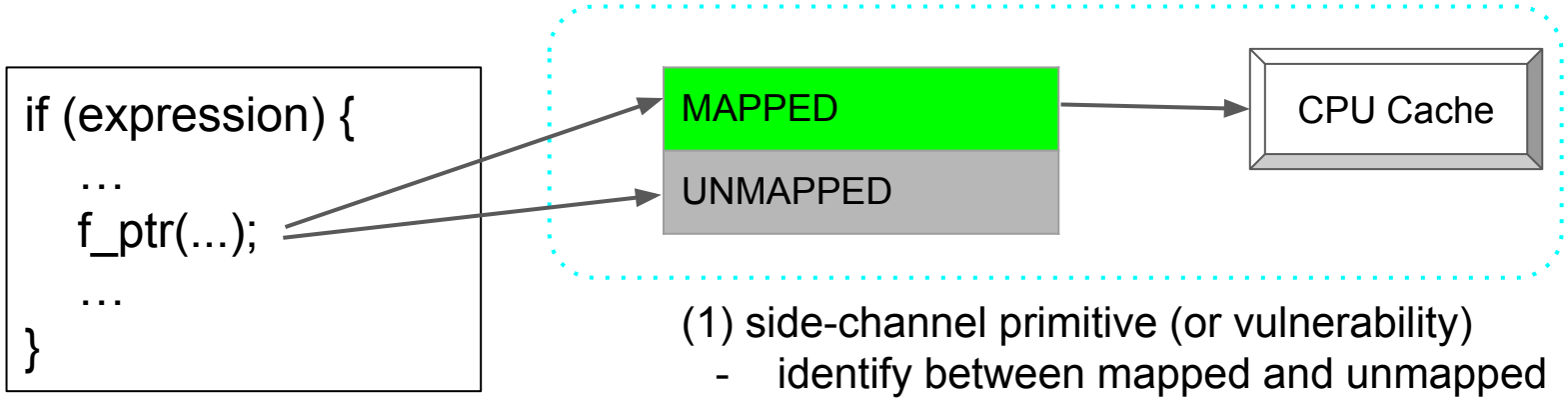
```
if (expression) {
    …
    val = *obj;
    …
}
```

MTE

- The ability we have: modifying f_ptr (left) or obj (right)
- The objective: bypassing PAC (left) or MTE (right) by brute-force
- The problem: putting in a invalid pointer resulting in a crash
- The solution: speculatively executing the if block!

Side-channel attacks

Two components of Speculative Probing (breaking ASLR)

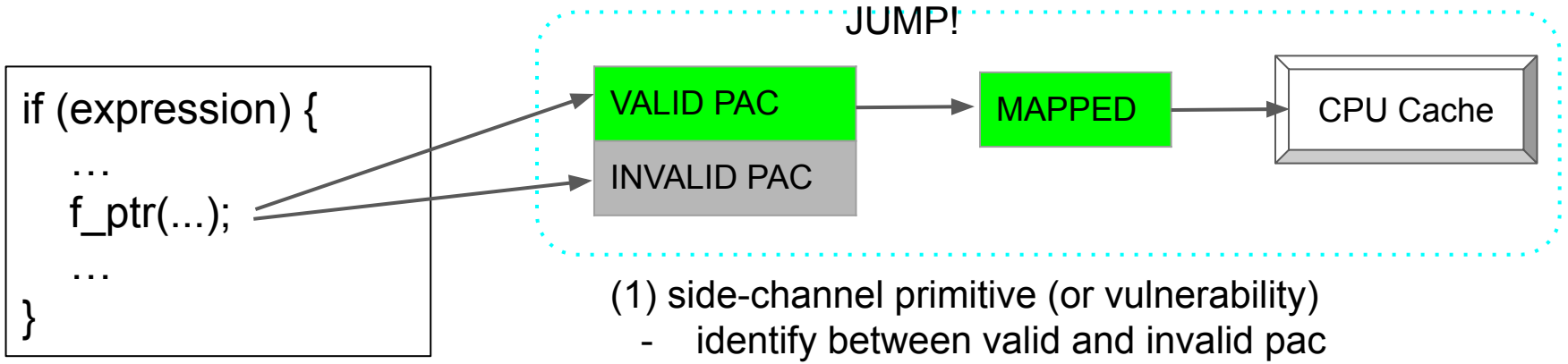


(1) side-channel primitive (or vulnerability)
- identify between mapped and unmapped

(2) exploit techniques
(speculative execution, cache measurement, …)

Side-channel attacks

Two components of Speculative Probing (breaking PAC)
(PACMAN, ISCA 2022)



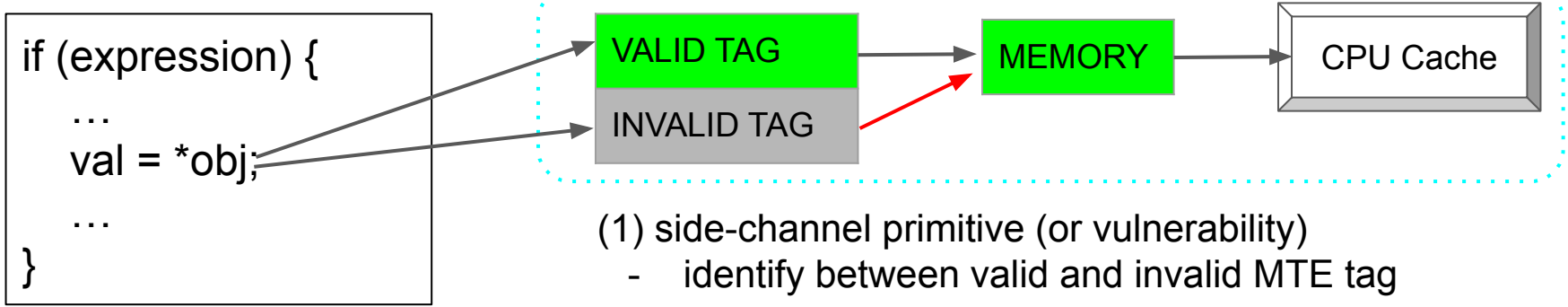
(1) side-channel primitive (or vulnerability)
- identify between valid and invalid pac

(2) exploit techniques
(speculative execution, cache measurement, …)

Side-channel attacks

Two components of Speculative Probing (breaking MTE)

Unfortunately, it doesn't work…. in case of MTE..



(1) side-channel primitive (or vulnerability)
- identify between valid and invalid MTE tag

(2) exploit techniques
(speculative execution, cache measurement, …)

Side-channel attacks

MTE as Implemented (Google Project Zero blog)

```
if (expression) {
    …
    ptr = *obj;
    val = *ptr;
}
```

What P0 did is a dependent access

- obj has a valid tag → ptr is accessed
- obj has an invalid tag → ptr is not accessed

Result? → ptr is accessed in both cases…
They concluded no side channel vuls.

Side-channel attacks

A lesson learned in terms of side channel primitives

PAC ✔

```
if (expression) {
    …
    f_ptr(...);
    …
}
```

MTE ✘

```
if (expression) {
    …
    val = *obj;
    …
}
```

MTE ✔

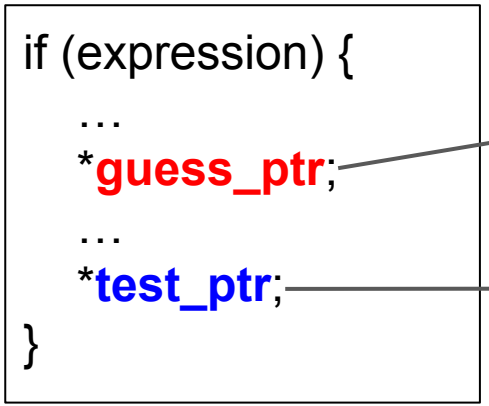
```
if (expression) {
    …
    val = *obj;
    *obj…;
}
```

- **PAC**: A single line of code (function pointer call) is enough to be a side-channel primitive.
- **MTE**: A single line of code (load or store) is not enough to be a side-channel primitive… we need to discover **gadgets made of multiple lines of codes**!

Vulnerabilities (Primitives)

Terms: two types of pointers

A code pattern



guess_ptr (where MTE tag check happens, on which attackers perform brute-force)

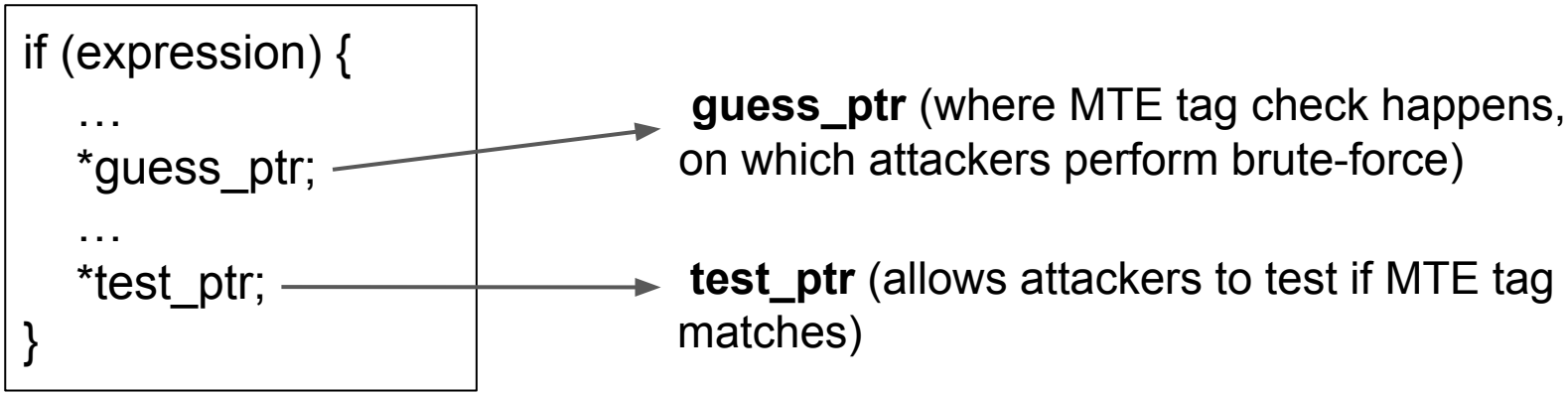
test_ptr (allows attackers to test if MTE tag matches)

Side-channel primitive (expected)?

- guess_ptr has a valid tag → test_ptr is accessed → test_ptr is in cache
- guess_ptr has an invalid tag → test_ptr is not accessed → test_ptr is not in cache

Our objective

A code pattern



Discover code patterns in which–
whether test_ptr is accessed depends on an MTE tag check result on guess_ptr!

V1 Gadget

TikTag V1 Gadget: the main insight

```
if (expression) {
    …
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
….
    val = *test_ptr;
}
```

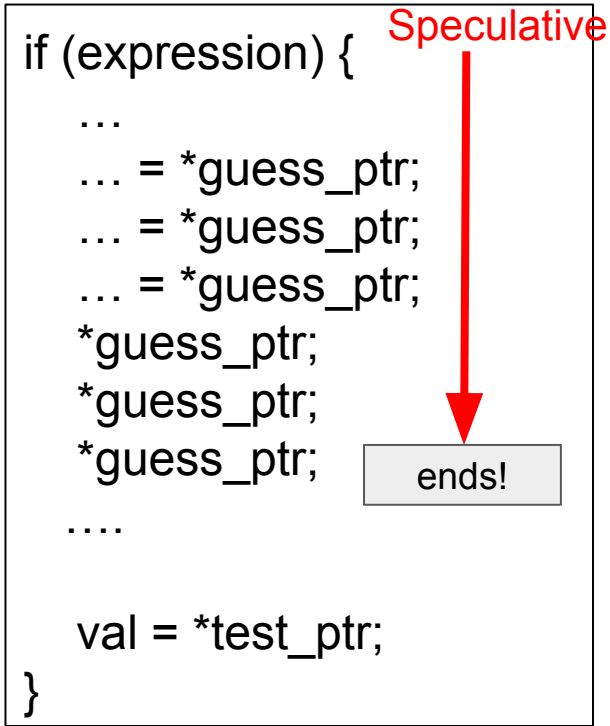


Main idea:

If a bunch of tag mismatch happens, CPU gets more contention internally (busy), leading to **test_ptr not accessed.**

V1 Gadget

TikTag V1 Gadget: the main insight



Note:
With too many guess_ptr accesses happen, it always couldn't get to test_ptr.
(no test_ptr access in case of tag match)

–

N guess_ptr accesses (i.e., if it has an invalid tag, it causes **N** tag mismatches)

V1 Gadget

TikTag V1 Gadget: 1st experiment and its problem

```
if (expression) {
    …
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
    val = *test_ptr;
}
```



On Pixel 8, **30-40** guess_ptr accesses exhibit the difference of test_ptr access. (valid → accessed, invalid → not accessed)

Problem?
30-40 accesses to the same object in a row is an unrealistic pattern… (unlikely to exist in a real SW)

V1 Gadget

TikTag V1 Gadget: the definitive form

```
BR:
if (expression) {
    …
CHECK:
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
GAP:
    nop
    …
TEST:
    val = *test_ptr;
}
```

The definitive form:

(1) BR: branch to mistrain
(2) CHECK: **N** guess_ptr accesses (smaller N)
(3) GAP: Make a gap (**T** cycles, using NOP)
(4) TEST: test_ptr access

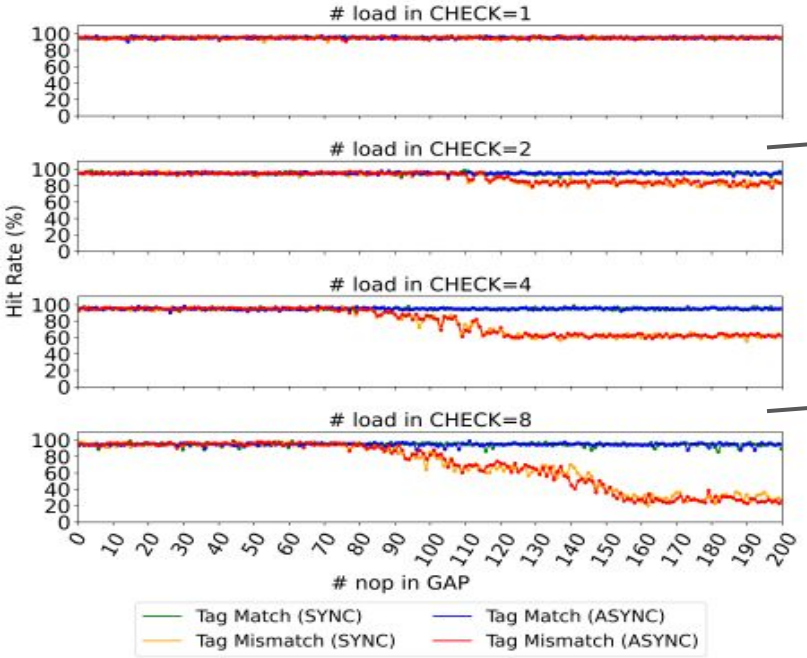
Experiment?

- Figuring out N and T.

V1 Gadget

TikTag V1 Gadget: experiment

test_ptr hit rate



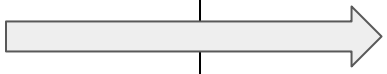
Less than 2 guess_ptr accesses is not enough…

More than or equal to **4** guess_ptr and **80** cycles of GAP gets this gadget to work!

V1 Gadget

TikTag V1 Gadget: the definitive conditions

```
BR:
if (expression) {
    …
CHECK:
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
GAP:
    nop
    …
TEST:
    val = *test_ptr;
}
```



More than or equal to 4 guess_ptr accesses!



More than 80 cycles between CHECK and TEST!
It can be any instructions!

V1 Gadget

TikTag V1 Gadget: a quick look at a real gadget

```
// BR: speculation branch with cond_ptr (tu)
switch (tu->tread) {
 case TREAD_FORMAT_TIME32: // branch prediction destination
    // CHECK: dereference guess_ptr (tread) with 4 loads
    tread32 = (struct snd_timer_tread32) {
         .event = tread->event,
         .tstamp_sec = tread->tstamp_sec,
         .tstamp_nsec = tread->tstamp_nsec,
         .val = tread->val,
     };
   // TEST: dereference test_ptr (buffer)
   if (copy_to_user(buffer, &tread32, sizeof(tr
         err = -EFAULT;
     break;
default: // correct branch destination
     err = -ENOTSUPP;
     break;
}
```

BR: Switch: branch to mistrain

CHECK:
tread: guess_ptr
4 guess_ptr accesses!

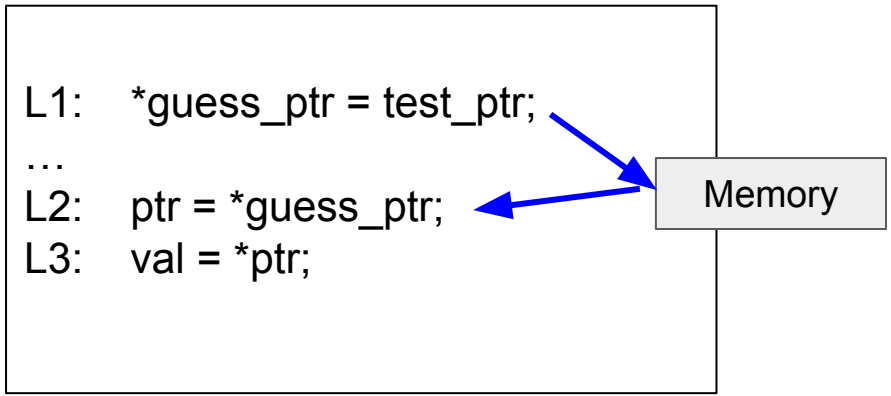
TEST:
buffer: test_ptr

More than 80 cycles in GAP because of many check codes in copy_to_user() before accessing buffer!

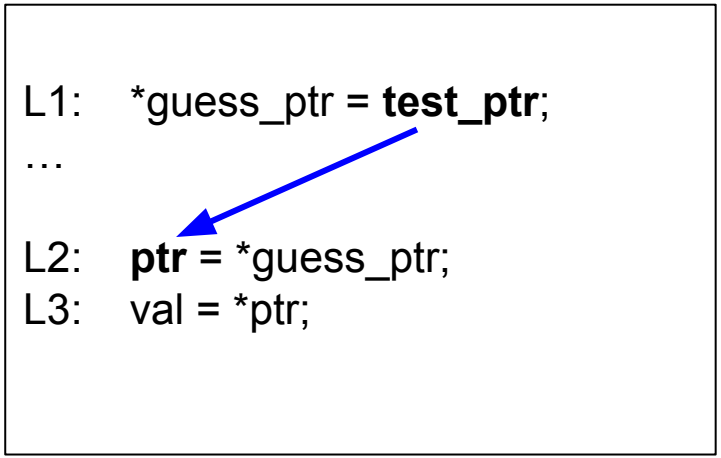
V2 Gadget

Background: store-to-load forwarding

What is store-to-load forwarding? (exploited in Spectre v4)



An unoptimized flow



An optimized flow
(store-to-load forwarding)

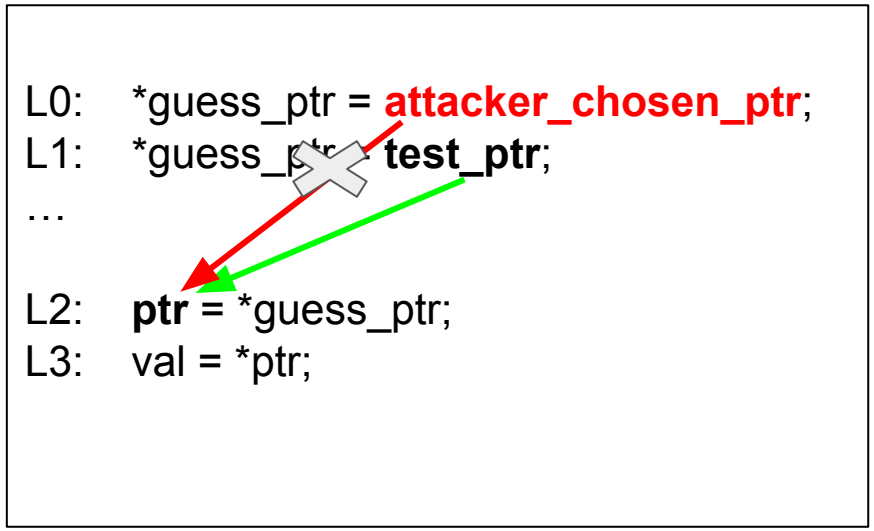
V2 Gadget

Background: Spectre v4 & SSBS (Speculative Store Bypass Safe)

```
L0:   *guess_ptr = attacker_chosen_ptr;
L1:   *guess_ptr = test_ptr;
…

L2:   ptr = *guess_ptr;
L3:   val = *ptr;
```

Spectre v4 in short
(accessing attacker_chosen_ptr at L3)



ARM's mitigation:
SSBS (Speculative Store Bypass Safe)

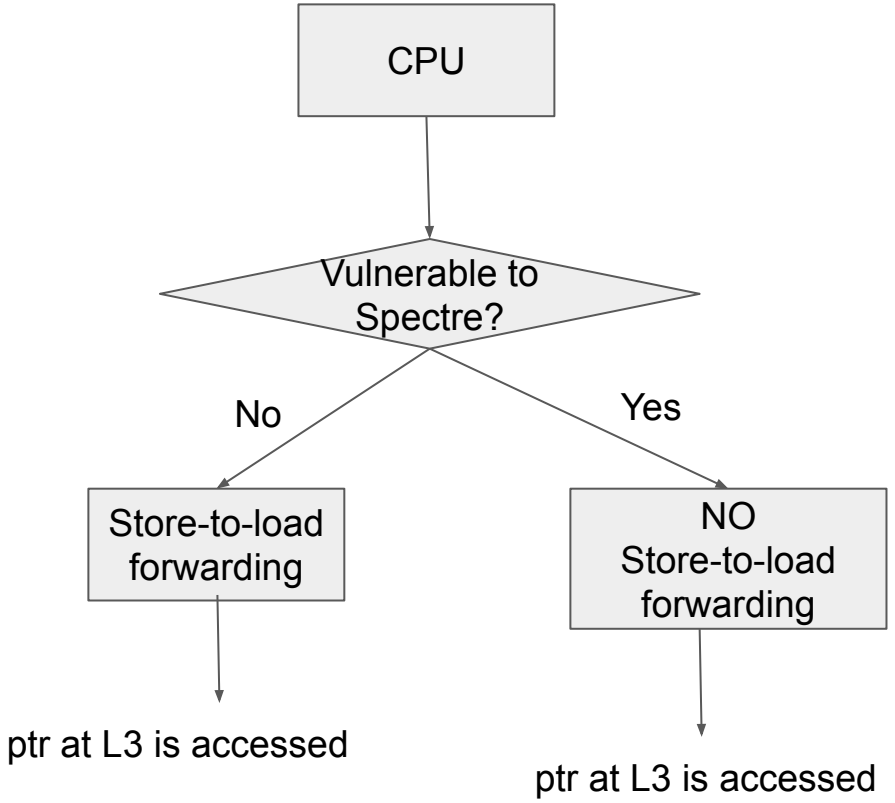
V2 Gadget

TikTag V2 Gadget: the key observation

```
L1:   *guess_ptr = test_ptr;
…

L2:   ptr = *guess_ptr;
L3:   val = *ptr;
```

SSBS without MTE



V2 Gadget

TikTag V2 Gadget: the key observation

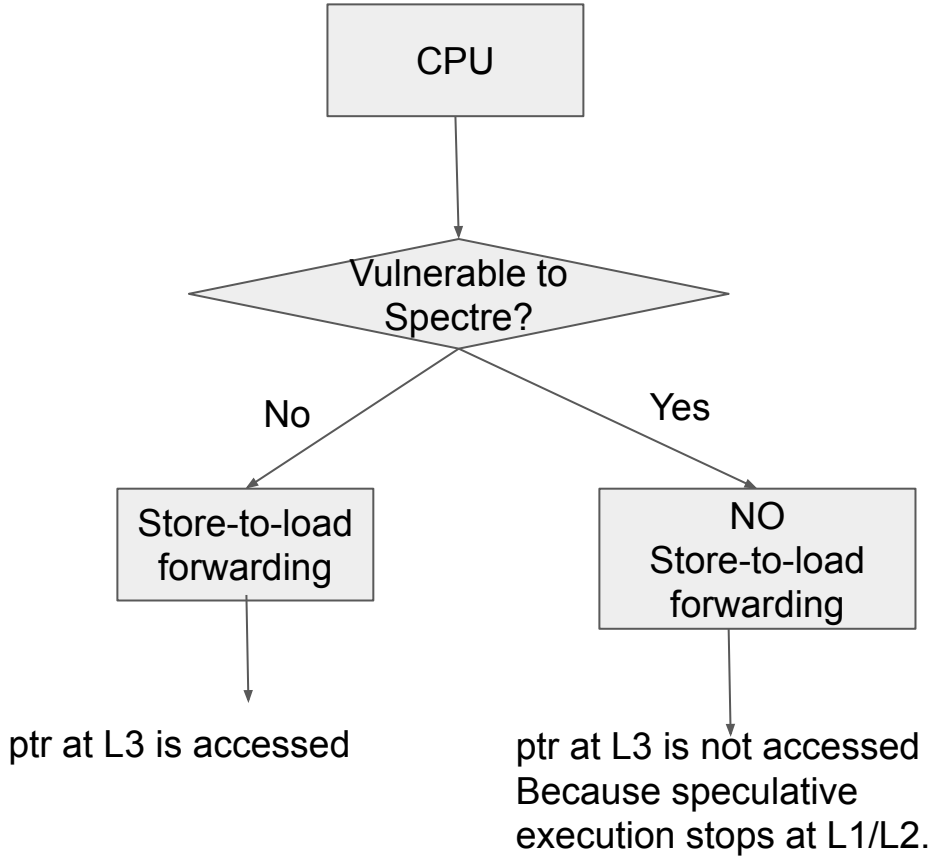
```
If (expression) {
   L1:    *guess_ptr = test_ptr;
   …

   L2:    ptr = *guess_ptr;
   L3:    val = *ptr;
}
```

SPECULATIVE EXECUTION!

SSBS itself is a side-channel primitive!

SSBS without MTE



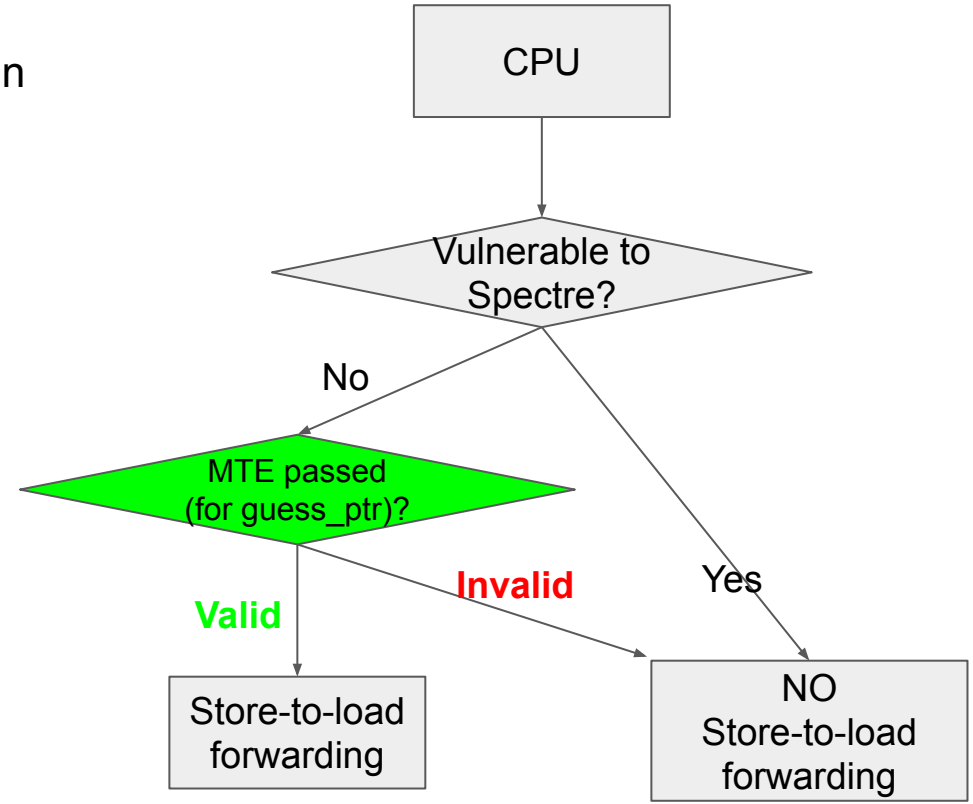
V2 Gadget

TikTag V2 Gadget: the key observation

```
L1:   *guess_ptr = test_ptr;
…

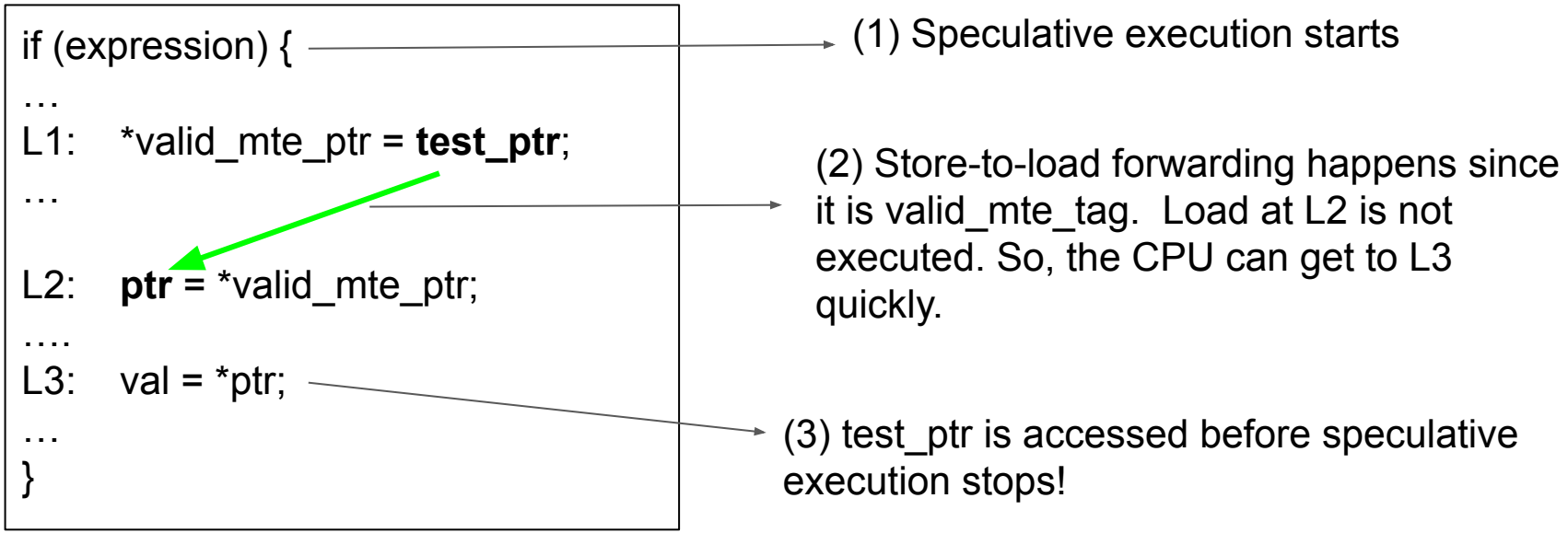
L2:   ptr = *guess_ptr;
L3:   val = *ptr;
```

SSBS with MTE



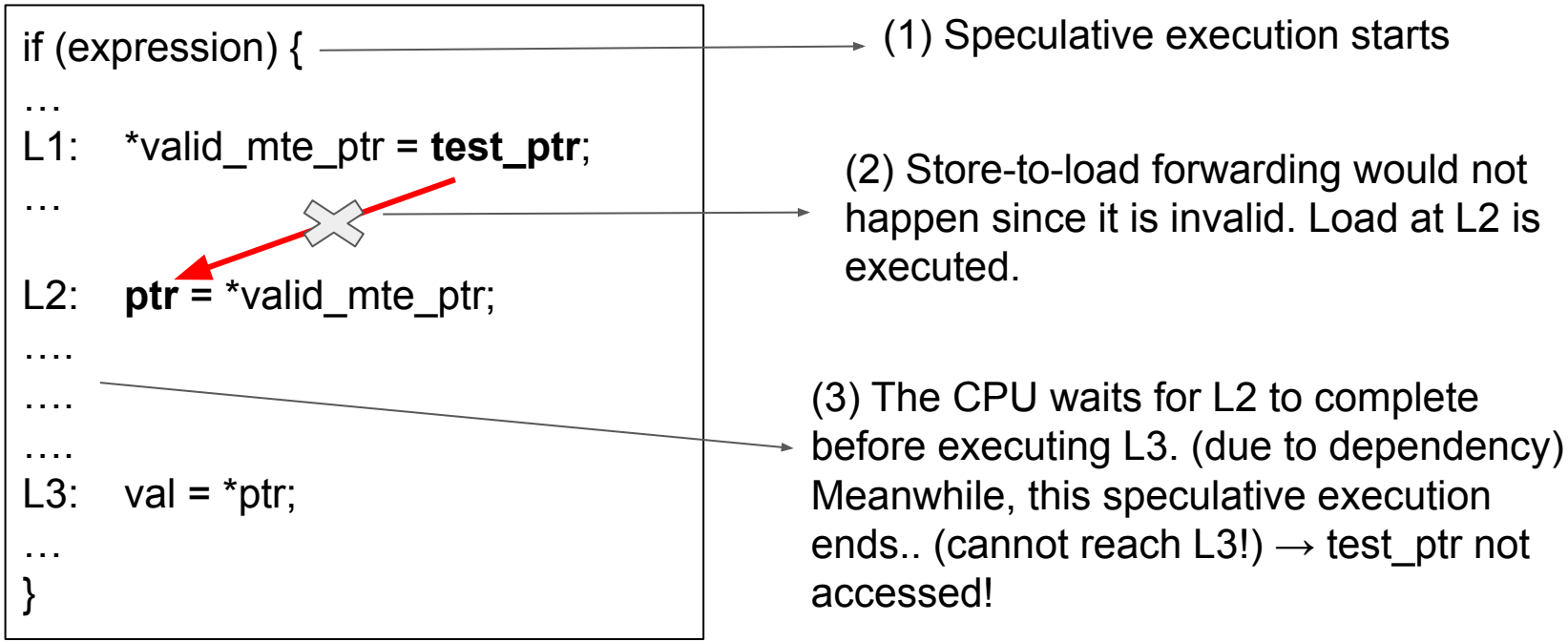
V2 Gadget

TikTag V2 Gadget: how it works (valid MTE tag)



V2 Gadget

TikTag V2 Gadget: how it works (invalid MTE tag)



V2 Gadget

TikTag V2 Gadget: the definitive form

```
BR:
   if (expression) {
CHECK:
   L1:   *guess_ptr = test_ptr;
   ….
   L2:   ptr = *guess_ptr;
TEST:
   L3:   val = *ptr;
}
```

Condition:
there must be less than or equal to 3 instructions between L1 and L2. (on pixel 8)
(related to a CPU's instruction window)

Exploitation: overview

Overview

- In our paper, we demonstrated bypassing MTE in two environments– Chrome browser and Linux kernel. The focus of this talk is Linux kernel.
- Two different ways to obtain an exploitable gadget (V1 or V2)
    - Gadget discovery: finding gadgets residing in the linux kernel and exploiting them
    - Gadget generation: crafting gadgets using a JIT compiler such as eBPF and exploiting them

- In our paper, we did “gadget discovery” on linux kernel and did “gadget generation on chrome browser.
- A realistic threat model (in linux)
    - A state-of-the art CFI defense is applied. (CFI bypass is impossible)
    - User-space attackers. (What attackers can only do is invoke system calls without root privilege)

What does “Exploitation” mean?

1. MTE is applied on the linux kernel you want to attack.
2. You have a kernel UAF vulnerability (or OOB) and exploit it successfully.
3. However, accessing a dangling pointer would cause a fault due to MTE tag mismatch.
4. What you want here is–
   a. **Testing if MTE tag matches or not, using our gadgets. → The key to bypass MTE**
   b. Access a dangling pointer only when MTE tag matches. (No fault!)

Exploitation in general

Exploitation in general

An example kernel driver that contains an UAF vul and a V1 Gadget

An example

Attacker process

User

```
*cond_ptr = user_val;
```

syscall-1

```
if (*cond_ptr != 0) {
    …
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
    …
    val = *test_ptr;
}
```

Object

An example V1 Gadget (syscall-2)

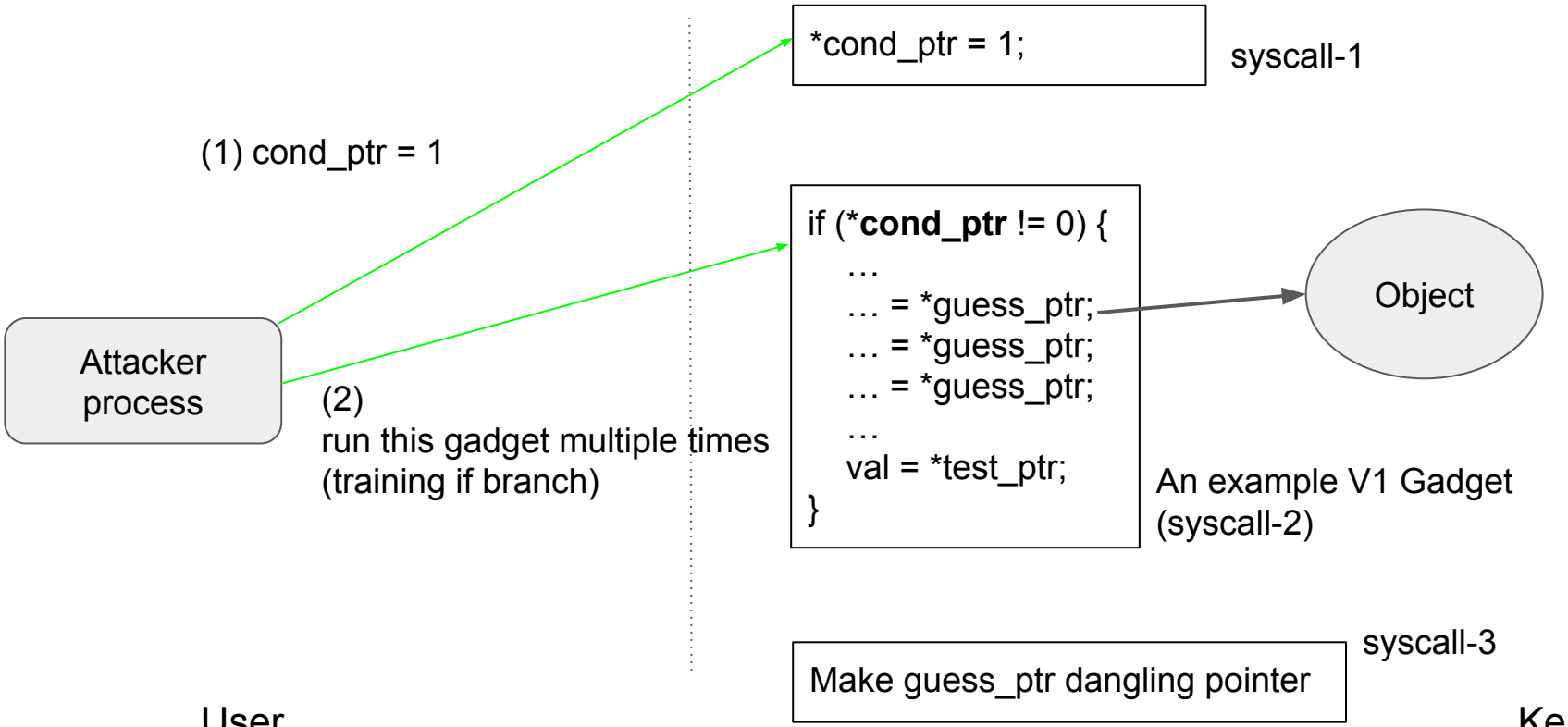
```
Make guess_ptr dangling pointer
```

syscall-3

Kernel

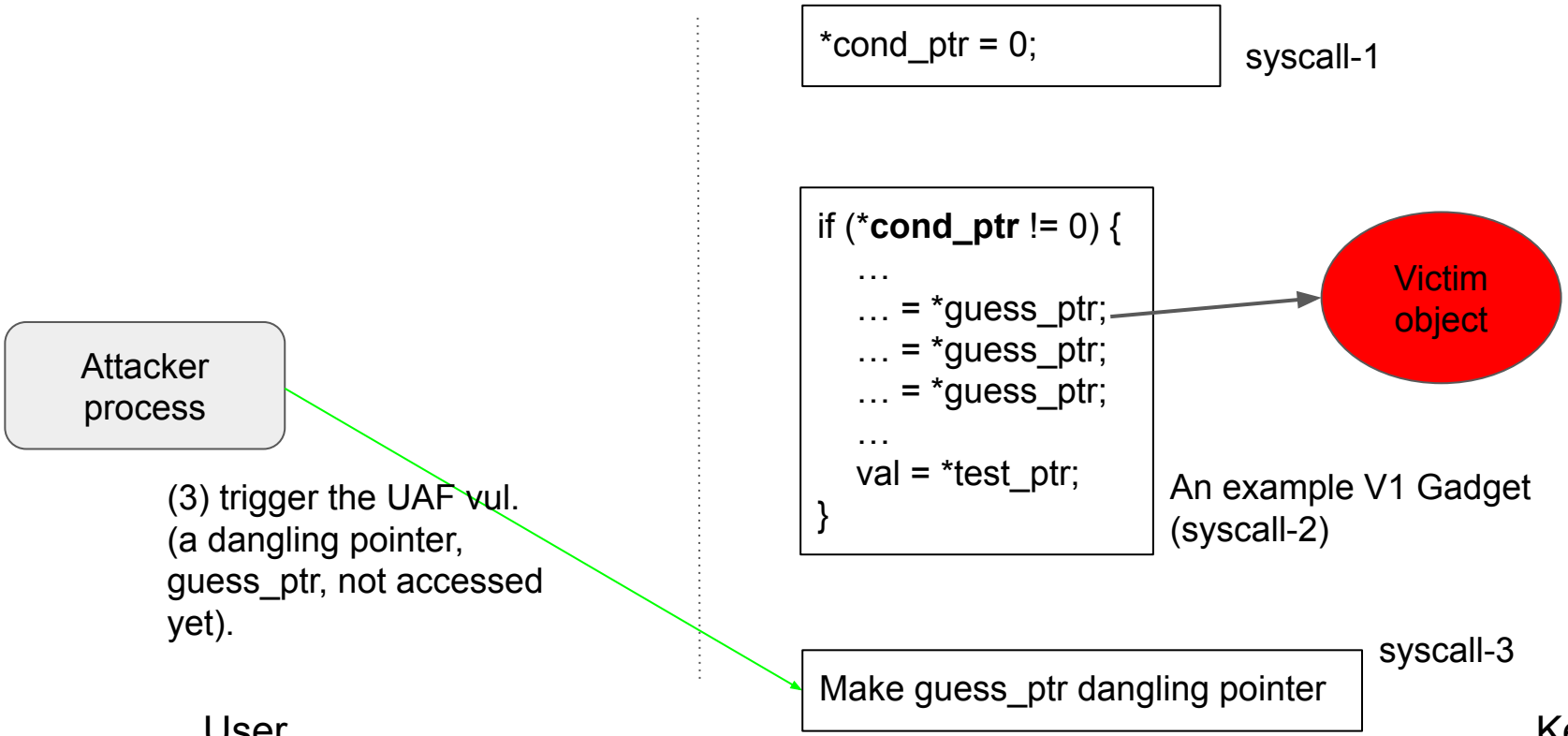
Exploitation in general

An example kernel driver that contains an UAF vul and V1 Gadget



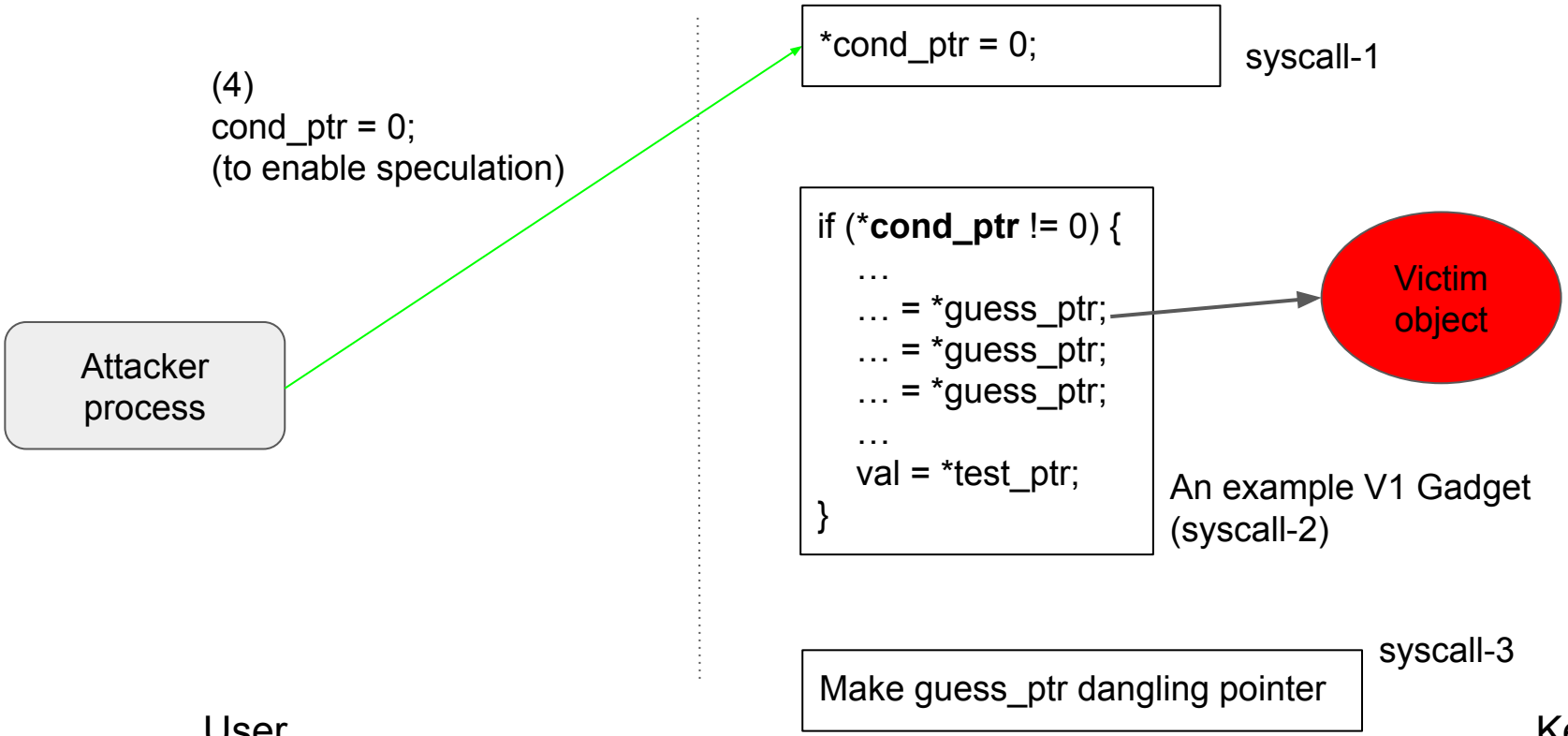
Exploitation in general

An example kernel driver that contains an UAF vul and V1 Gadget



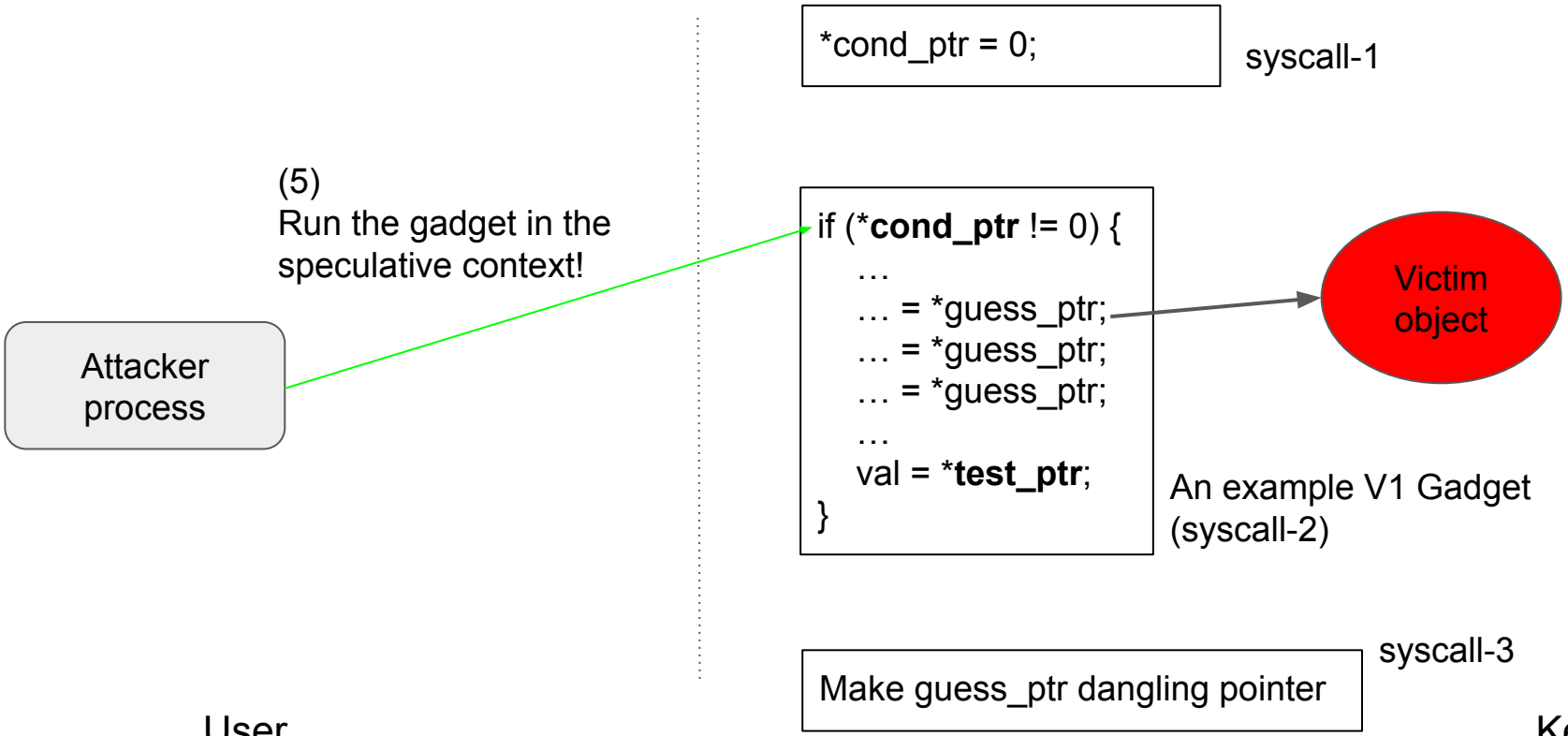
Exploitation in general

An example kernel driver that contains an UAF vul and V1 Gadget



Exploitation in general

An example kernel driver that contains an UAF vul and V1 Gadget



Exploitation in general

An example kernel driver that contains an UAF vul and V1 Gadget

```
*cond_ptr = 0;
```

syscall-1

(6)
Measure test_ptr!
test_ptr is in cache only when MTE tag matches!

Attacker process

```
if (*cond_ptr != 0) {
    …
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
    …
    val = *test_ptr;
}
```

Victim object

An example V1 Gadget (syscall-2)

Tag match → do a dangling pointer access
Tag mismatch → start all over again

```
Make guess_ptr dangling pointer
```

syscall-3

User

Kernel

Exploitation in general

Two techniques (cache control) needed for successful exploitations:

T1: How to cache-flush cond_ptr from user-space before executing the gadget speculatively?

T2: How to cache-flush test_ptr and measure the elapsed time of test_ptr from user-space?

User

An example kernel driver that contains an UAF vul and V1 Gadget

```
*cond_ptr = 0;
```

syscall-1

```
if (*cond_ptr != 0) {
    …
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
    …
    val = *test_ptr;
}
```



An example V1 Gadget (syscall-2)

```
Make guess_ptr dangling pointer
```

syscall-3

Kernel

Exploitation in general

An example kernel driver that contains an UAF vul and V1 Gadget

T1: cache-flush cond_ptr

Attacker process

Cache-flush directly?
Not work..

```
*cond_ptr = 0;
```

syscall-1

```
if (*cond_ptr != 0) {
    …
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
    …
    val = *test_ptr;
}
```

An example V1 Gadget (syscall-2)

```
Make guess_ptr dangling pointer
```

syscall-3

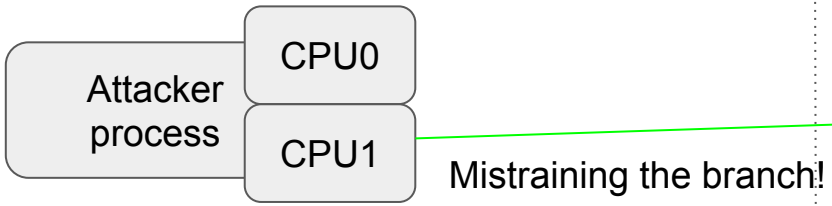
User

Kernel

Exploitation in general

An example kernel driver that contains an UAF vul and V1 Gadget

T1: cache-flush cond_ptr (cache line bouncing)



Mistraining the branch!

```
if (*cond_ptr != 0) {
    …
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
    …
    val = *test_ptr;
}
```

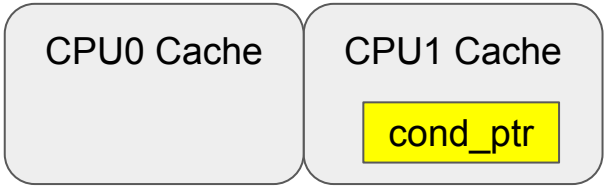
An example V1 Gadget (syscall-2)

```
*cond_ptr = 0;
```

syscall-1

```
Make guess_ptr dangling pointer
```

syscall-3



User

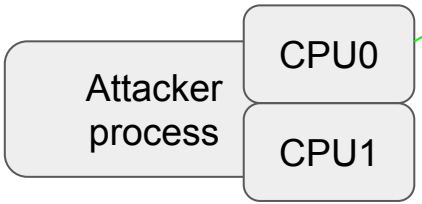
Kernel

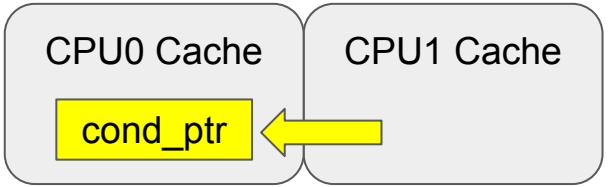
Exploitation in general

An example kernel driver that contains an UAF vul and V1 Gadget

T1: cache-flush cond_ptr (cache line bouncing)

Access cond_ptr in a different CPU and syscall!





User

```
*cond_ptr = 0;
```

syscall-1

```
if (*cond_ptr != 0) {
    …
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
    …
    val = *test_ptr;
}
```

An example V1 Gadget (syscall-2)

Make guess_ptr dangling pointer

syscall-3

Kernel

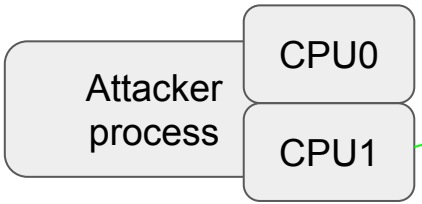
Exploitation in general

An example kernel driver that contains an UAF vul and V1 Gadget

T1: cache-flush cond_ptr (cache line bouncing)

```
*cond_ptr = 0;
```

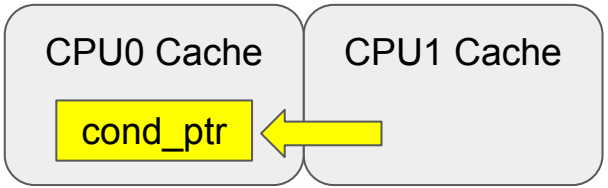
syscall-1



Speculatively execute the gadget!

```
if (*cond_ptr != 0) {
    …
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
    …
    val = *test_ptr;
}
```

An example V1 Gadget (syscall-2)




```
Make guess_ptr dangling pointer
```

syscall-3

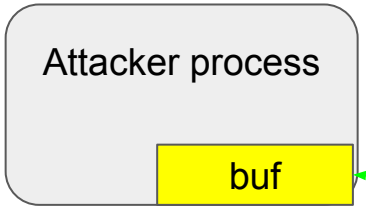
User

Kernel

Exploitation in general

An example kernel driver that contains an UAF vul and V1 Gadget

T2: Measure test_ptr (using user-space test_ptr)



We assume test_ptr points to the user-space buffer to make things easier to measure its cache state.

```
*cond_ptr = 0;
```

syscall-1

```
if (*cond_ptr != 0) {
    …
    … = *guess_ptr;
    … = *guess_ptr;
    … = *guess_ptr;
    …
    copy_to_user(test_ptr, ..)
}
```

An example V1 Gadget (syscall-2)

```
Make guess_ptr dangling pointer
```

syscall-3

User

Kernel

Gadget discovery

What exploitable kernel V1 gadgets look like

We made a simple gadget scanner. (an imprecise static analysis)

```
if (*cond_ptr != 0) {
    …
    … = obj->v1;   // guess_ptr
    … = obj->v2;   // guess_ptr
    … = obj->v3;   // guess_ptr
    … = obj->v4;   // guess_ptr
    …
    copy_to_user(test_ptr, ..)
}
```

(1) It detects “if/switch branch” that involves a load instruction
(2) In the branch, there has to be more than 4 loads pointing to the same object (having the same MTE tag) → More number of loads are better to exploit.
(3) In the branch, there must be a copy_to/from_user() call.

Digging..

- ~ 100 potential gadgets found (in Pixel8 kernel built with a default configuration)
- Code-review them one by one manually → filter out non-exploitable ones
- Prioritize potentially exploitable ones, based on the number of guess_ptr loads. (many is better)
- Exploit 20+ gadgets in reality on Pixel 8.

A confirmed gadget

```
// BR: speculation branch with cond_ptr (tu)
switch (tu->tread) {
 case TREAD_FORMAT_TIME32: // branch prediction destination
    // CHECK: dereference guess_ptr (tread) with 4 loads
    tread32 = (struct snd_timer_tread32) {
         .event = tread->event,
         .tstamp_sec = tread->tstamp_sec,
         .tstamp_nsec = tread->tstamp_nsec,
         .val = tread->val,
     };
   // TEST: dereference test_ptr (buffer)
   if (copy_to_user(buffer, &tread32, sizeof(tread32)))
         err = -EFAULT;
     break;
default: // correct branch destination
     err = -ENOTSUPP;
     break;
}
```

tu->tread: cond_ptr

tread: guess_ptr
(4 loads)

buffer: test_ptr

- V1 gadget in snd_timer_user_read()
- We injected an artificial UAF vulnerability that makes “tread” dangling pointer, and successfully exploited this gadget along with it.
- This was the only gadget worked out.. out of 20+ gadgets I tried..

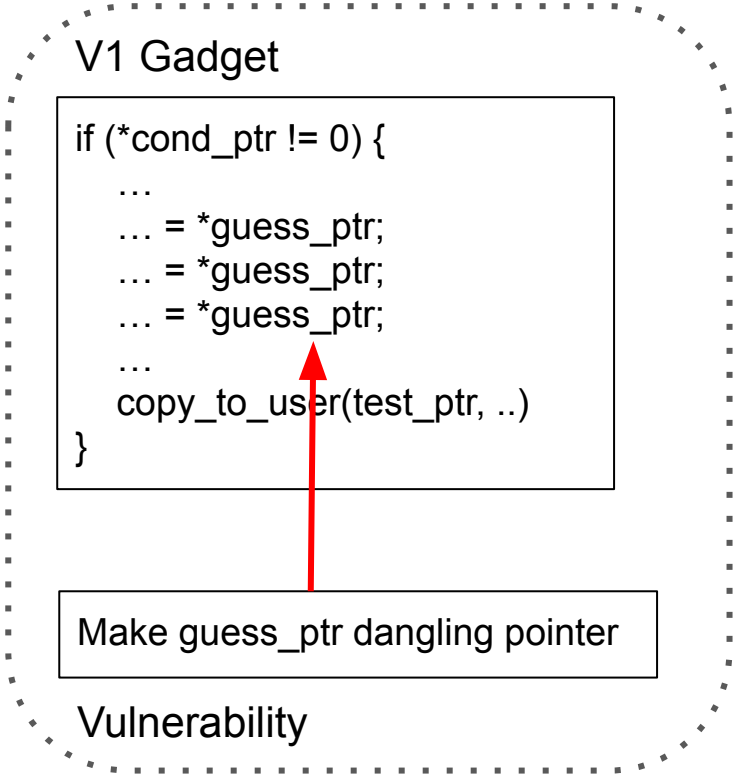
One critical limitation in real gadget exploitation

ONE CRITICAL LIMITATION!

You have a vulnerability. And to exploit it, there must be a V1 Gadget nearby the vulnerability. (V1 Gadget and Vulnerability are tightly coupled..)

Is it realistic?? → very unlikely…

We investigated a number of CVEs but none of them has an exploitable V1 gadget..

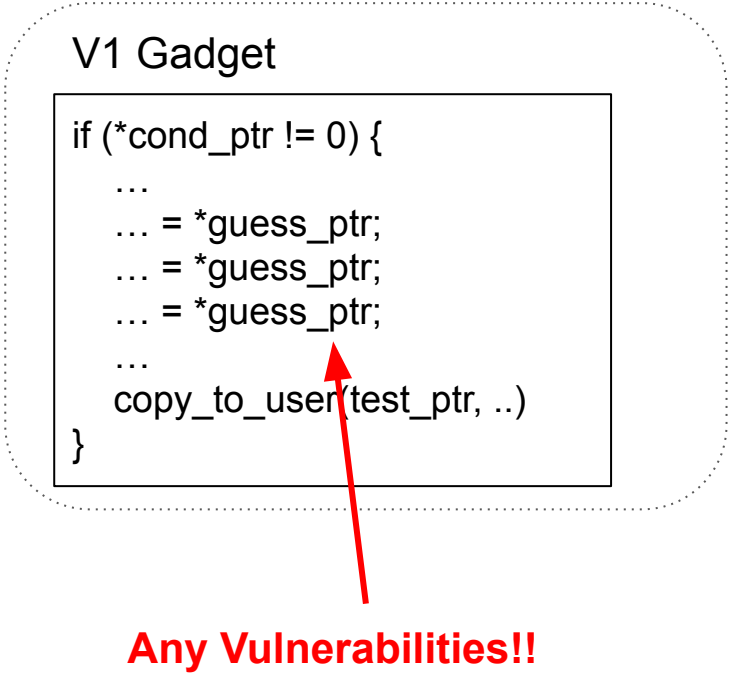


One critical limitation in real gadget exploitation

What we want to do?

When you have an exploitable V1 Gadget, you want to exploit this gadget for any vulnerabilities that has nothing to do with the Gadget.
(decouple the two)

How can we do this?



Double-free to the rescue

vuln_ptr == guess_ptr == victim_ptr

Flow-1 (Vulnerability)

free(vuln_ptr);

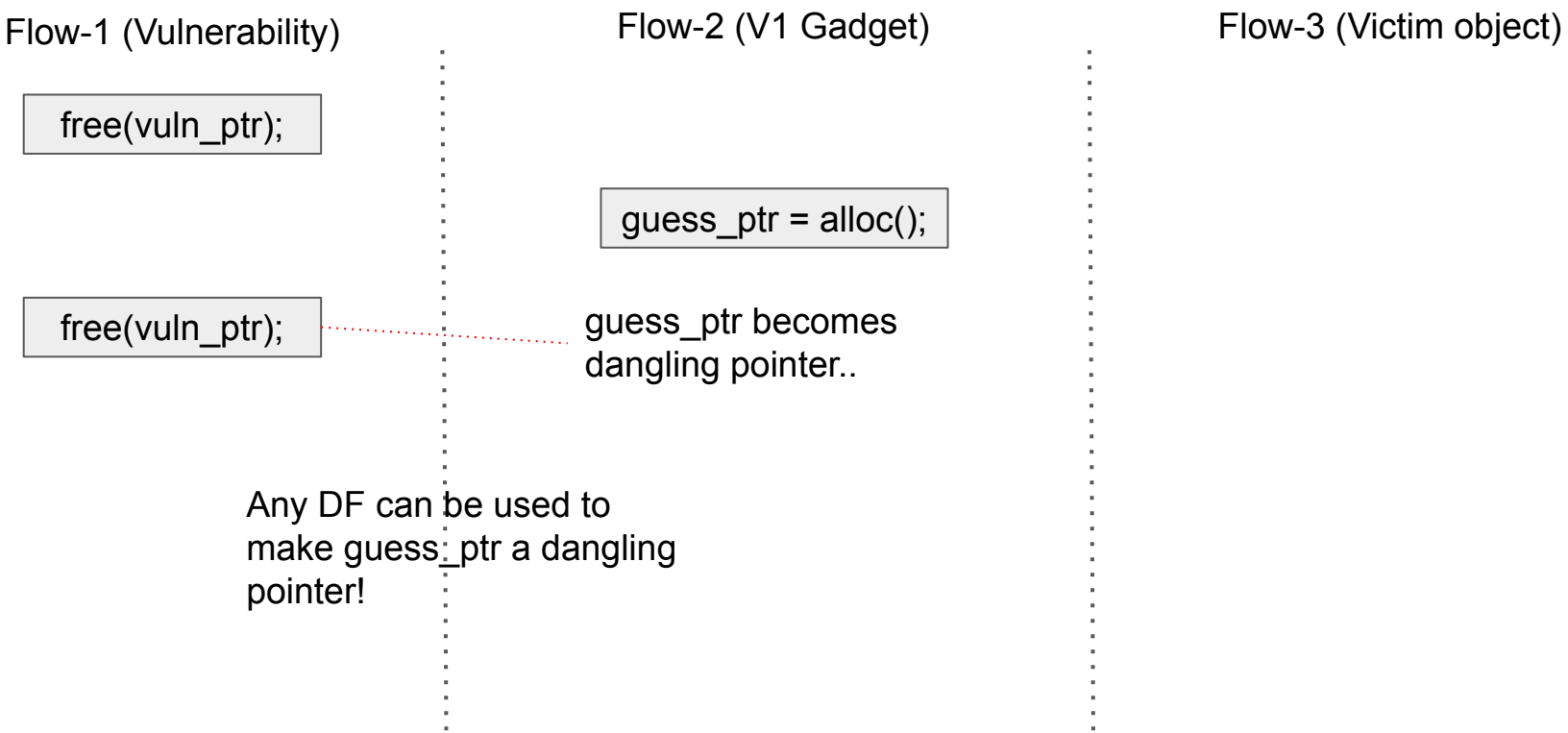
Flow-2 (V1 Gadget)

guess_ptr = alloc();

Flow-3 (Victim object)

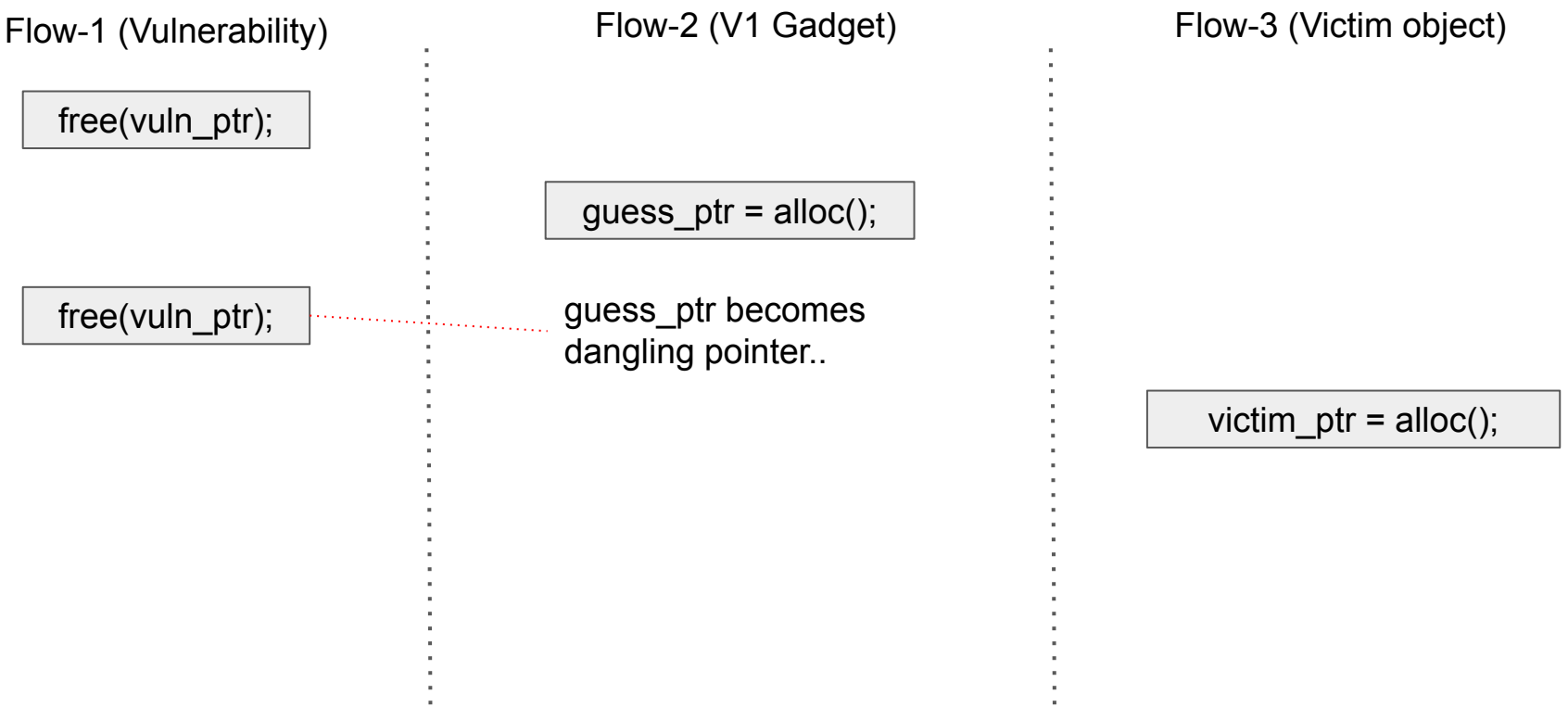
Double-free to the rescue

vuln_ptr == guess_ptr == victim_ptr



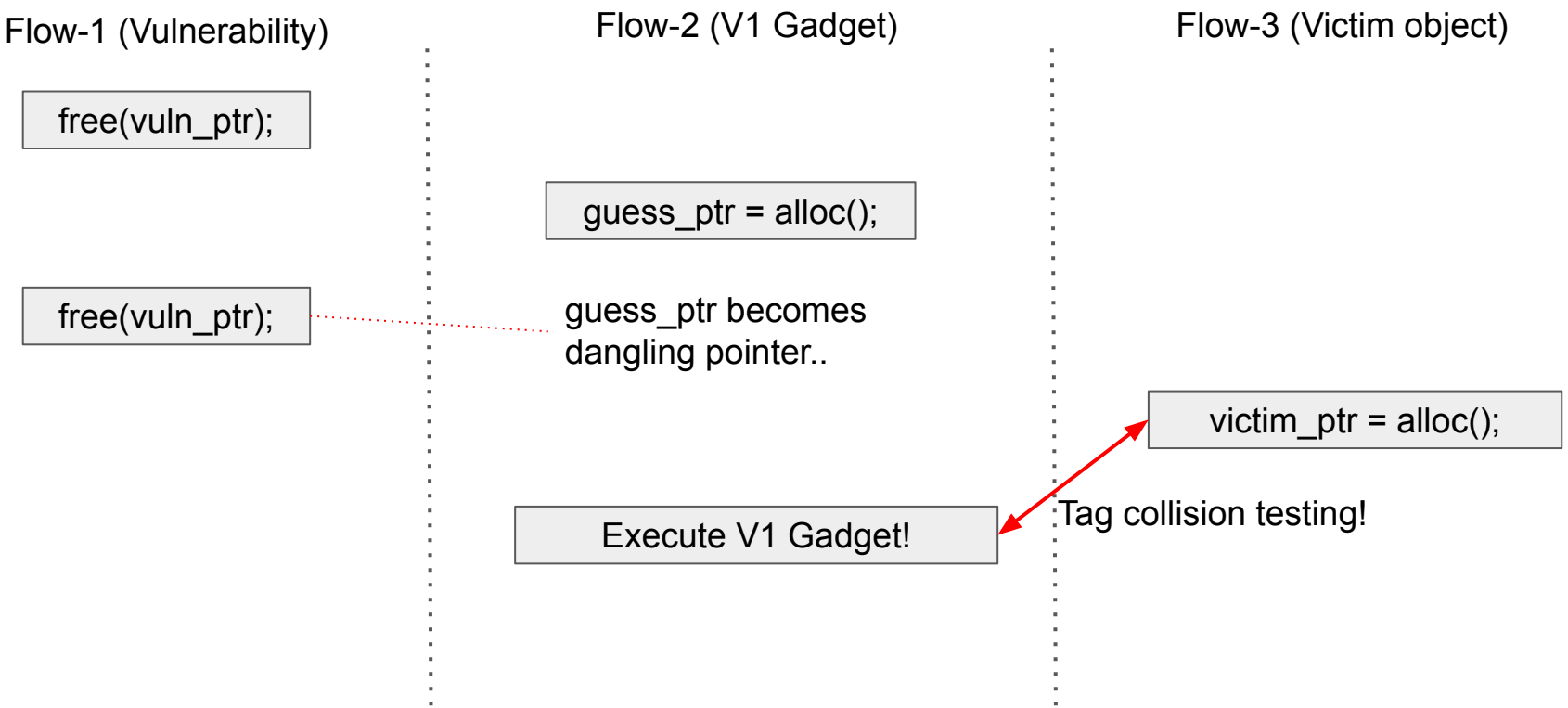
Double-free to the rescue

vuln_ptr == guess_ptr == victim_ptr



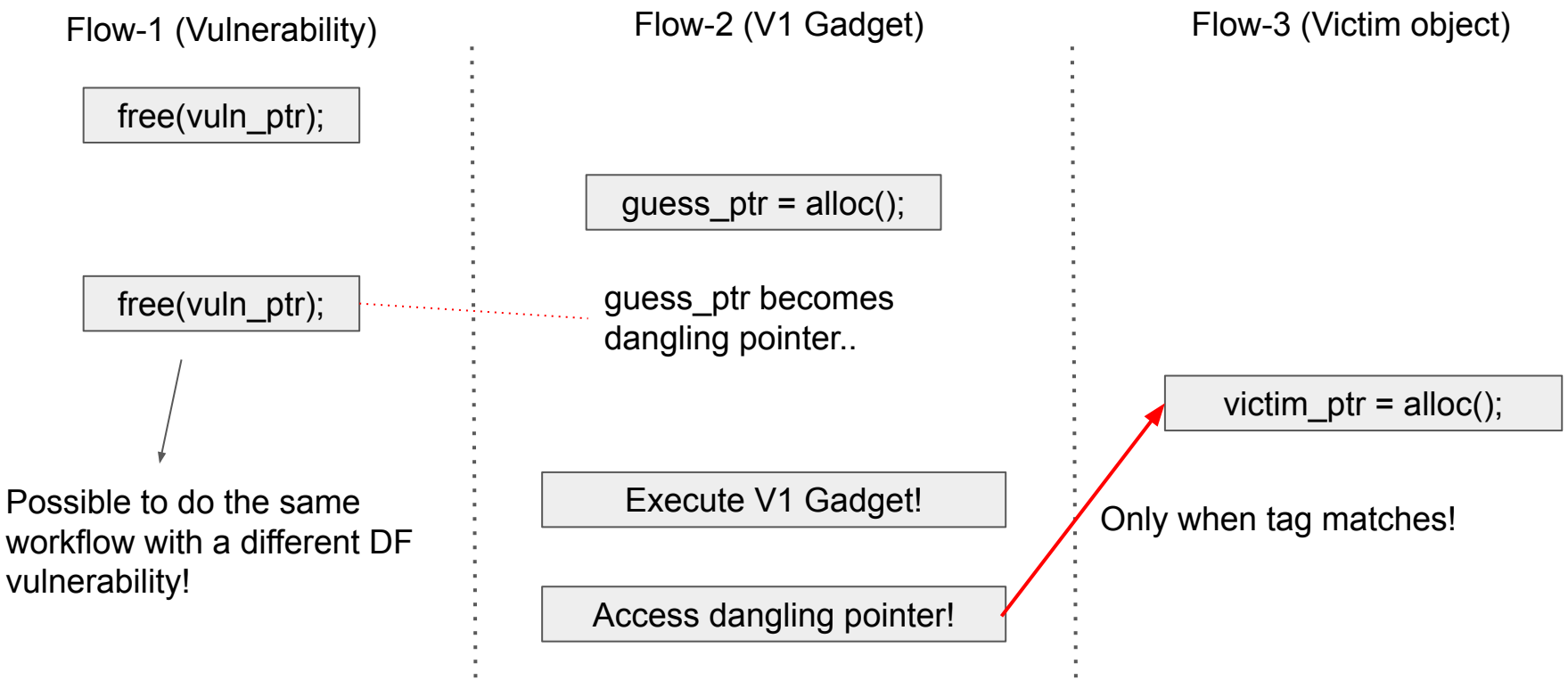
Double-free to the rescue

vuln_ptr == guess_ptr == victim_ptr

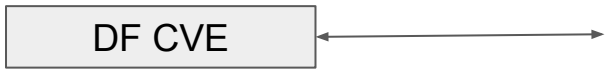


Double-free to the rescue

vuln_ptr == guess_ptr == victim_ptr



Double-free to the rescue



A real V1 gadget

```
// BR: speculation branch with cond_ptr (tu)
switch (tu->tread) {
 case TREAD_FORMAT_TIME32: // branch prediction destination
    // CHECK: dereference guess_ptr (tread) with 4 loads
    tread32 = (struct snd_timer_tread32) {
         .event = tread->event,
         .tstamp_sec = tread->tstamp_sec,
         .tstamp_nsec = tread->tstamp_nsec,
         .val = tread->val,
     };
   // TEST: dereference test_ptr (buffer)
   if (copy_to_user(buffer, &tread32, sizeof(tread32)))
        err = -EFAULT;
     break;
default: // correct branch destination
     err = -ENOTSUPP;
     break;
}
```

We confirmed that a combination of one DF CVE and the V1 gadget allowed us to access a dangling pointer under the radar of MTE!

–

Demonstrated the practicality of our attack!

A wrap up for V1 gadget

- There are many potential V1 gadgets, but working gadget is rare… (in our experiment, only 1 working gadget confirmed out of 20+ gadgets)

- A pretty big amount of engineering is needed to test if a gadget works in reality.. (some automation would be helpful here. e.g., fuzzing to determine if it works or not)

- If your vulnerability is not DF, it's very unlikely you can exploit it along with a V1 gadget in real SW. But if you have an exploitable DF, the likelihood would increase.

V2 Gadget

V2 Gadget in general

```
L1:   *guess_ptr = test_ptr;
L2:   ptr = *guess_ptr;
L3:   val = *ptr;
```

Long story short:
There was no working V2 Gadget in the linux kernel… (few potential gadgets though)

(exclude details..)

Gadget generation

Gadget generation

- Its focus is to generate a V2 gadget using eBPF and exploit it.

- This part is not included in our paper because recent mobile android platforms don’t allow normal android apps to load a custom eBPF program… (so we concluded there is no practical security implication)
  - In our paper, gadget generation is demonstrated in Chrome browser not linux.
  - So, I didn’t spend much time on this stuff..

- We did several attempts though (3 different approaches). We used a rooted Pixel8 in order to run a custom eBPF program that contains a V2 gadget.

- Will not detail here…

A wrap up for My Presentation

- MTE in nature has fewer SW-level vulnerabilities than PAC. (more challenging to bypass)
- So, launching a side-channel attack to bypass MTE is a promising direction.
- We found two different side-channel primitives (V1, V2) that can lead to MTE bypass.
- We confirmed one real gadget (V1) to work out on Pixel 8.
- We couldn’t locate real V2 gadgets on Pixel8. Instead, V2 gadgets can be crafted by JIT compilers and demonstrated in chrome browser not linux.

End